

PROJETO DE INTERFACES DO USUÁRIO

CONCEITOS-CHAVE

acessibilidade	305
análise de tarefas ...	295
análise de usuários ..	294
atribuição de nomes a comandos	304
avaliação de projetos	311
carga de memória ...	289
controle	288
elaboração de tarefas.	296
interface	
análise	294
consistência	290
modelos	291
projeto	300
internacionalização ..	305
princípios e diretrizes .	306
processo	292
projeto de interface para WebApp	306
recursos de ajuda ...	303
regras de ouro	288
tempo de resposta ..	302
tratamento de erros .	304
usabilidade	291

Vivemos em um mundo de produtos de alta tecnologia e praticamente todos — produtos eletrônicos de consumo, equipamentos industriais, sistemas corporativos, sistemas militares, software para PC e WebApps — requerem interação humana. Para que um produto de software seja bem-sucedido, deve apresentar boa usabilidade — medida qualitativa da facilidade e eficiência com a qual um ser humano consegue empregar as funções e os recursos oferecidos pelo produto de alta tecnologia.

Independentemente de uma interface ter sido projetada para um aparelho de música digital ou para um sistema de controle de armamento de um caça, a usabilidade importa. Se os mecanismos de interface tiverem sido bem projetados, o usuário flui suavemente através da interação usando um ritmo cadenciado que permite que o trabalho seja realizado sem grandes esforços. Entretanto, se a interface for mal concebida, o usuário se move aos trancos e barrancos, e o resultado será frustração e baixa eficiência no trabalho.

Nas primeiras três décadas da era computacional, a usabilidade não era uma preocupação dominante entre aqueles que construíam software. Em seu clássico livro sobre projeto, Donald Norman [Nor88] argumentou que já era tempo de uma mudança de atitude:

Para criar tecnologia que se adapte ao ser humano, é necessário estudá-lo. Mas hoje temos uma tendência de estudar apenas a tecnologia. Como consequência, exige-se que as pessoas se adaptem à tecnologia. É chegada a hora de inverter a tendência, a hora de fazer com que a tecnologia se adapte às pessoas.

À medida que os tecnólogos passaram a estudar a interação humana, surgiram duas questões preponderantes. Primeiro, identificou-se um conjunto de *regras de ouro* (discutidas na Seção 11.1). Estas se aplicavam a toda interação humana com produtos tecnológicos. Em segundo lugar, definiu-se um conjunto de *mecanismos de interação* para permitir aos projetistas de software construir sistemas que implementassem de forma apropriada

PANORAMA

O que é? O projeto de interfaces do usuário cria um meio de comunicação efetivo entre o ser humano e o computador. Seguindo-se um conjunto de princípios de projeto de interfaces, o projeto identifica objetos e ações de interface e então cria um layout de tela que forma a base para um protótipo de interface do usuário.

Quem realiza? Um engenheiro de software projeta a interface do usuário por meio da aplicação de um processo iterativo que faz uso de princípios de projeto predefinidos.

Por que é importante? Se um software for difícil de ser utilizado, se ele o compele a incorrer em erros ou frustra seus esforços de atingir suas metas, você não gostará dele, independentemente do poder computacional apresentado, do conteúdo fornecido ou da funcionalidade oferecida. A interface deve ser correta, pois molda a percepção do software pelo usuário.

Quais são as etapas envolvidas? O projeto de interfaces do usuário se inicia pela identificação do

usuário, das tarefas e dos requisitos do ambiente. Uma vez que as tarefas de usuário tenham sido identificadas, os cenários são criados e analisados para definir um conjunto de objetos e ações de interface. Os cenários formam a base para a criação do layout da tela que representa o projeto gráfico e o posicionamento de ícones, a definição de texto descritivo na tela, a especificação e os títulos de janelas, bem como a especificação de itens de menu principais e secundários. São usadas ferramentas para criar protótipos e, por fim, implementar o modelo de projeto e o resultado é avaliado em termos de qualidade.

Qual é o artefato? São criados cenários de usuário e gerados layouts de tela. É desenvolvido um protótipo de interface modificado de forma iterativa.

Como garantir que o trabalho foi realizado corretamente? Os usuários fazem um *test drive* da interface e o feedback do *test drive* é usado para a próxima modificação iterativa do protótipo.

as regras de ouro. Os mecanismos de interação, coletivamente denominado *interface gráfica do usuário* (*graphical user interface, GUI*), eliminaram parte da maioria dos atrozes problemas associados às interfaces humanas. Mas mesmo no “mundo Windows”, todos nós já encontramos interfaces do usuário que são difíceis de assimilar, difíceis de usar, confusas, contraintuitivas, implacáveis e, em muitos casos, totalmente frustrantes. Mesmo assim, alguns gastam tempo e energia construindo cada uma dessas interfaces e é pouco provável que seus construtores tenham criado os problemas propositadamente.

11.1 AS REGRAS DE OURO

Em seu livro sobre projeto de interfaces, Theo Mandel [Man97] cunha três *regras de ouro*:

1. Deixar o usuário no comando.
2. Reduzir a carga de memória do usuário.
3. Tornar a interface consistente.

Essas regras formam, na verdade, a base para um conjunto de princípios para o projeto de interfaces do usuário que orienta esse importante aspecto do projeto de software.

11.1.1 Deixar o usuário no comando

Durante uma sessão para levantamento de requisitos para um importante e novo sistema de informação, perguntou-se a um usuário-chave sobre os atributos da interface gráfica orientada a janelas.

“O que realmente gostaria”, disse o usuário solenemente, “é de um sistema que leia minha mente. Ele saberia o que eu quero fazer antes mesmo de eu ter de fazê-lo e isso me facilitaria tremendamente a vida. Isso é tudo, apenas isso.”

Minha primeira reação foi de sacudir a cabeça e sorrir, porém, refleti por um momento. Não havia absolutamente nada de errado com a solicitação do usuário. Ele queria um sistema que reagisse às suas necessidades e o ajudasse a concretizar suas tarefas. Ele queria controlar o computador, e não que o computador o controlasse.

A maioria das limitações e restrições de interface impostas por um projetista destina-se a simplificar o modo de interação. Mas para quem?

Como projetista, talvez sejamos tentados a introduzir restrições e limitações para simplificar a implementação da interface. O resultado poderia ser uma interface fácil de ser construída, mas frustrante sob o ponto de vista do usuário. Mandel [Man97] define uma série de princípios de projeto que permitem a um usuário manter o controle:

Defina modos de interação para não forçar o usuário a realizar ações desnecessárias ou indesejadas. Modo de interação é o estado atual da interface. Por exemplo, se for escolhido o comando de *correção ortográfica* no menu de um processador de texto, o software entra no modo de correção ortográfica. Não há nenhuma razão para forçar o usuário a permanecer no modo de revisão ortográfica se ele quer apenas fazer uma pequena edição de texto no meio do caminho. O usuário deve ser capaz de entrar e sair desse modo com pouco ou nenhum esforço.

Proporcione interação flexível. Pelo fato de diferentes usuários terem preferências de interação diversas, deve-se fornecer opções. Por exemplo, um software poderia permitir a um usuário interagir por meio de comandos de teclado, movimentação do mouse, caneta digitalizadora, tela multitoque ou por comandos de reconhecimento de voz. Mas nem toda ação é suscetível a todo mecanismo de interação. Considere, por exemplo, a dificuldade de usar comandos via teclado (ou entrada de voz) para desenhar uma forma complexa.

Possibilite que a interação de usuário possa ser interrompida e desfeita. Mesmo quando envolvido em uma sequência de ações, o usuário deve ser capaz de interromper a sequência para fazer alguma outra coisa (sem perder o trabalho que já havia feito). O usuário também deve ser capaz de “desfazer” qualquer ação.

“É melhor projetar a experiência do usuário do que retificá-la.”

Jon Meads

“Sempre desejei que meu computador fosse tão fácil de ser usado quanto meu telefone. Meu desejo tornou-se realidade. Já não sei mais como usar meu telefone.”

Bjarne Stroustrup
(criador do C++)

Simplifique a interação à medida que os níveis de competência avançam e permita que a interação possa ser personalizada. Geralmente os usuários constatarem que realizam a mesma sequência de interações repetidamente. Vale a pena criar um mecanismo de “macros” que permita a um usuário avançado personalizar a interface para facilitar sua interação.

Oculte os detalhes técnicos de funcionamento interno do usuário casual. Uma interface com o usuário deve levá-lo ao mundo virtual da aplicação. O usuário não deve se preocupar com o sistema operacional, as funções de arquivos ou alguma outra tecnologia computacional enigmática. Em essência, a interface jamais deve exigir que o usuário interaja em um nível “interno” à máquina (por exemplo, jamais se deve exigir que um usuário tenha de digitar comandos do sistema operacional a partir do ambiente da aplicação).

Projete para interação direta com objetos que aparecem na tela. O usuário tem uma sensação de controle quando lhe é permitido manipular os objetos necessários para realizar uma tarefa de maneira similar ao que ocorreria caso o objeto fosse algo físico. Por exemplo, uma interface de aplicação que permita a um usuário “esticar” um objeto (aumentar ou diminuir em escala o seu tamanho) é uma implementação de manipulação direta.

11.1.2 Reduzir a carga de memória do usuário

Quanto mais um usuário tiver de se lembrar, mais sujeita a erros será a interação com o sistema. É por essa razão que uma interface do usuário bem desenhada não exaure a memória do usuário. Sempre que possível, o sistema deve “se lembrar” de informações pertinentes e auxiliar o usuário em um cenário de interação que o ajude a recordar-se. Mandel [Man97] define princípios de projeto que possibilitam a uma interface reduzir a carga de memória do usuário:

Reduza a demanda de memória recente. Quando os usuários estão envolvidos em tarefas complexas, a demanda de memória recente pode ser significativa. A interface deve ser projetada para reduzir a exigência de recordar ações, entradas e resultados passados. Isso pode ser obtido pelo fornecimento de pistas visuais que permitam a um usuário reconhecer ações passadas, em vez de ter de se recordar delas.

Estabeleça defaults significativos. O conjunto de parâmetros iniciais (defaults) deve fazer sentido para o usuário comum, porém um usuário também deve ser capaz de especificar suas preferências individuais. Entretanto, deve-se fornecer uma opção “reset”, que permita o restabelecimento dos valores-padrão originais.

Defina atalhos intuitivos. Quando forem usados mnemônicos para realizar alguma função de sistema (por exemplo, alt-P para chamar a função de impressão), esse mnemônico deve estar ligado à ação de uma forma que seja fácil de ser memorizada (por exemplo, a primeira letra da tarefa a ser solicitada).

O layout visual da interface deve se basear na metáfora do mundo real. Por exemplo, um sistema de pagamento de contas deve usar uma metáfora de talão de cheques e registro de cheques para orientar o usuário pelo processo de pagamento de uma conta. Isso permite ao usuário que se apoie em indicações visuais bem compreensíveis, em vez de ter de memorizar uma sequência de interações misteriosa.

Revele as informações de maneira progressiva. A interface deve ser organizada hierarquicamente. As informações sobre uma tarefa, um objeto ou algum comportamento devem ser apresentadas inicialmente em um alto nível de abstração. Mais detalhes devem ser apresentados após o usuário demonstrar interesse por meio de uma seleção com o mouse. Um exemplo, comum a muitas aplicações de processamento de texto, é a função de sublinhado. A função em si é uma de uma série de funções agrupadas em um menu de *estilo de texto*. Entretanto, todos os recursos de sublinhado não são apresentados logo de cara. O usuário tem de selecionar a função de sublinhado; em seguida, todas as opções de sublinhado (por exemplo, sublinhado simples, duplo, tracejado) são apresentadas.

CASA SEGURA



Violação de uma regra de ouro de uma interface do usuário

Cena: Sala do Vinod, quando é iniciado o projeto da interface do usuário.

Atores: Vinod e Jamie, membros da equipe de engenharia de software do CasaSegura.

Conversa:

Jamie: Estive pensando sobre a interface da função de vigilância.

Vinod (sorrindo): Pensar faz bem.

Jamie: Creio que talvez possamos simplificar um pouco as coisas.

Vinod: O que você quer dizer?

Jamie: Bem, que tal se eliminássemos totalmente a planta. Ela é “atraente”, porém irá nos dar muito trabalho de projeto. Em vez disso, poderíamos apenas solicitar ao usuário especificar a câmera que deseja ver e então exibir o vídeo em uma janela de vídeo.

Vinod: Como o proprietário irá se lembrar de quantas câmeras estão configuradas e onde se encontram?

Jamie (levemente irritado): Ele é o proprietário do imóvel; ele deve saber essas coisas.

Vinod: Mas e se não souber?

Jamie: Ele deve.

Vinod: Essa não é a questão... E se ele se esquecer?

Jamie: Poderíamos fornecer-lhe uma lista de câmeras em operação e suas posições.

Vinod: Isso é possível, mas por que ele deveria ter de solicitar uma lista?

Jamie: Certo, nós fornecemos a lista se ele pedir ou não.

Vinod: Melhor. Pelo menos não terá de se lembrar de coisas que podemos dar a ele.

Jamie (pensando por um instante): Mas você gosta da planta, não é mesmo?

Vinod: Sim.

Jamie: Qual delas você acha que o pessoal de marketing irá gostar?

Vinod: Tá brincando, não é mesmo?

Jamie: Não.

Vinod: Sei lá... Aquela com efeito piscante... Eles adoram características chamativas para o produto... Eles não estão interessados em qual é a mais fácil de ser construída.

Jamie (suspirando): Certo, talvez possamos fazer um protótipo para ambas.

Vinod: Boa ideia... Depois deixamos o cliente decidir.

11.1.3 Tornar a interface consistente

“Coisas que parecem diferentes deveriam agir distintamente. Coisas que parecem iguais deveriam agir da mesma forma.”

Larry Marine

A interface deve apresentar e obter informações de forma consistente. Isso implica: (1) todas as informações visuais são organizadas de acordo com regras de projeto mantidas ao longo de todas as exibições de telas, (2) mecanismos de entrada são restritos a um conjunto limitado que é usado de forma consistente por toda a aplicação e (3) mecanismos de navegação para passar de uma tarefa a outra são definidos e implementados de maneira consistente. Mandel [Man97] define um conjunto de princípios de projeto que ajudam a tornar a interface consistente:

Permita ao usuário inserir a tarefa atual em um contexto significativo. Muitas interfaces implementam camadas de interações complexas com dezenas de imagens de tela. É importante fornecer indicadores (por exemplo, títulos para as janelas, ícones gráficos, sistema de cores consistente) que possibilitem ao usuário saber o contexto do trabalho em mãos. Além disso, o usuário deve ser capaz de determinar de onde ele veio e quais alternativas existem para transição para uma nova tarefa.

Mantenha a consistência ao longo de uma família de aplicações. Um conjunto de aplicações (ou produtos) deve implementar as mesmas regras de projeto de modo que a consistência seja mantida para toda a interação.

Se modelos interativos anteriores tiverem criado expectativa nos usuários, não faça alterações a menos que haja uma forte razão para isso. Uma vez que determinada sequência interativa tenha se tornado um padrão de fato (por exemplo, o uso de alt-S para salvar um arquivo), o usuário pressupõe que isso vá ocorrer em qualquer aplicação que vá usar. Uma mudança (por exemplo, usar alt-S para chamar uma função de escala) irá causar confusão.

Os princípios de projeto para interfaces discutidos nesta e em seções anteriores dão uma orientação básica. Nas seções seguintes, veremos o processo de projeto de interfaces em si.

INFORMAÇÕES

**Usabilidade**

Em um artigo esclarecedor sobre usabilidade, Larry Constantine [Con95] faz uma pergunta que tem uma relevância significativa sobre o tema: “O que os usuários querem, afinal de contas?”. Ele responde da seguinte maneira:

O que os usuários realmente querem são boas ferramentas. Todos os sistemas de software, de sistemas operacionais e linguagens a aplicações de entrada de dados e de apoio à decisão, são apenas ferramentas. Os usuários finais querem das ferramentas que criamos para eles praticamente o mesmo que esperamos das ferramentas que utilizamos. Eles querem sistemas fáceis de aprender e que os ajude a realizar seu trabalho. Querem software que não os retarde, não os engane ou confunda, que não facilite a prática de erros ou dificulte a finalização de seus trabalhos.

Constantine argumenta que a usabilidade não é derivada da estética, mecanismos de interação de última geração ou de inteligência incorporada às interfaces. Ao contrário, ocorre quando a arquitetura da interface atende às necessidades das pessoas que a usarão.

Uma definição formal de usabilidade é um tanto ilusória. Donahue e seus colegas [Don99] a definem da seguinte maneira: “Usabilidade é uma medida do quanto um sistema computacional... Facilita o aprendizado; ajuda os aprendizes a se lembrarem daquilo que aprenderam; reduz a probabilidade de erros; permite que se tornem eficientes e os deixa satisfeitos com o sistema”.

A única maneira de determinar se existe ou não “usabilidade” em um sistema que estamos construindo é realizar a ava-

liação ou teste de usabilidade. Observe os usuários interagirem com o sistema e faça-lhes as seguintes perguntas [Con95]:

- O sistema é utilizável sem a necessidade de ajuda ou aprendizado contínuo?
- As regras de interação ajudam um usuário experiente a trabalhar eficientemente?
- Os mecanismos de interação se tornam mais flexíveis à medida que os usuários se tornam mais capacitados?
- O sistema foi ajustado para o ambiente físico e social em que será usado?
- O usuário está ciente do estado do sistema? O usuário sempre sabe onde se encontra?
- A interface é estruturada de maneira lógica e consistente?
- Os mecanismos de interação, ícones e procedimentos são consistentes por toda a interface?
- A interação antecipa erros e ajuda o usuário a corrigi-los?
- A interface é tolerante com erros cometidos?
- A interação é simples?

Se cada uma dessas questões for respondida “positivamente”, é provável que a usabilidade tenha sido atingida.

Entre os muitos benefícios mensuráveis obtidos de um sistema utilizável, temos [Don99]: aumento nas vendas e na satisfação do cliente, vantagem competitiva, melhores avaliações por parte da mídia, melhor recomendação boca a boca, menores custos de suporte, aumento da produtividade do usuário final, redução nos custos de treinamento e de documentação, menor probabilidade de litígio com clientes descontentes.

11.2 ANÁLISE E PROJETO DE INTERFACES

WebRef

Uma excelente fonte de informações sobre projeto de interfaces do usuário pode ser encontrada em www.useit.com.

O processo geral para análise e projeto de uma interface do usuário se inicia com a criação de diferentes modelos de funções do sistema (segundo uma percepção do mundo externo). Começamos pelo delineamento das tarefas orientadas à interação homem-máquina necessárias para alcançar a função do sistema e, em seguida, consideramos as questões de projeto que se aplicam a todos os projetos de interface. São usadas ferramentas para prototipagem e, por fim, a implementação do modelo de projeto para o resultado ser avaliado pelos usuários finais em termos de qualidade.

11.2.1 Modelos de análise e projeto de interfaces

Quatro modelos distintos entram em cena ao se analisar e projetar uma interface do usuário. Um engenheiro humano (ou o engenheiro de software) estabelece um *modelo de usuário*, o engenheiro de software cria um *modelo de projeto*, o usuário final desenvolve uma imagem mental em geral chamada *modelo mental* do usuário ou *percepção do sistema* e os implementadores do sistema criam um *modelo de implementação*. Infelizmente, cada um dos modelos pode diferir significativamente. O papel de um projetista de interfaces é reconciliar as diferenças e obter uma representação consistente da interface.

O modelo de usuário estabelece o perfil dos usuários finais do sistema. Em sua coluna introdutória sobre “projeto centralizado no usuário”, Jeff Patton [Pat07] observa o seguinte:

“Se há um ‘truque’ para isso, a interface do usuário está quebrada.”

Douglas
Anderson



Até mesmo um usuário novato quer atalhos; mesmo usuários frequentes e com conhecimentos algumas vezes precisam de orientação. Dê a eles aquilo de que precisam.

PONTO-CHAVE

O modelo mental do usuário molda como o usuário percebe a interface e se a interface do usuário atende às suas necessidades.

“Preste atenção naquilo que os usuários fazem e não no que dizem.”

Jakob Nielsen

A verdade é que, projetistas e desenvolvedores — até eu mesmo — em geral pensam nos usuários. Entretanto, na ausência de um consistente modelo mental de usuário específico, nos colocamos no lugar desses usuários. A autossustituição não é centralizada no usuário — é egocêntrica.

Para construir uma interface do usuário efetiva, “todo projeto deve começar com um entendimento dos usuários pretendidos, incluindo seus perfis de idade, gênero, habilidades físicas, educação, formação cultural ou origem étnica, motivação, metas e personalidade” [Shn04]. Além disso, os usuários podem ser classificados como:

Novatos. Nenhum conhecimento sintático¹ do sistema e pouco conhecimento semântico² da aplicação ou uso do computador em geral.

Usuários intermitentes e com conhecimento. Razoável conhecimento semântico da aplicação, mas lembrança relativamente baixa das informações sintáticas necessárias para usar a interface.

Usuários frequentes e com conhecimento. Bom conhecimento semântico e sintático que muitas vezes levam à “síndrome do usuário poderoso”; indivíduos que procuram atalhos e modos de interação abreviados.

O *modelo mental* de usuário (percepção do sistema) é a imagem do sistema que os usuários finais trazem em suas mentes. Por exemplo, se fosse perguntado ao usuário de determinado processador de texto para descrever sua operação, a percepção do sistema orientaria a resposta. A precisão da descrição irá depender do perfil do usuário (por exemplo, novatos dariam no máximo uma resposta muito superficial) e da familiaridade geral com o software no domínio de aplicação. Um usuário que entenda completamente de processadores de texto, mas que trabalhou com um processador de texto específico apenas uma vez, talvez, na verdade, fosse capaz de dar uma descrição mais completa de sua função do que o novato que investiu semanas tentando aprender o sistema.

O *modelo de implementação* combina a manifestação externa do sistema computacional (a aparência e a percepção da interface), acoplada a todas as informações de apoio (livros, manuais, fitas de vídeo, arquivos de ajuda) que descrevem a sintaxe e a semântica da interface. Quando o modelo de implementação e o modelo mental de usuário são coincidentes, em geral os usuários sentem-se à vontade com o software e o utilizam de maneira eficaz. Para conseguir tal “amalgama” dos modelos, o modelo de projeto deve ter sido desenvolvido para levar em conta as informações contidas no modelo de usuário, e o modelo de implementação deve refletir precisamente as informações sintáticas e semânticas sobre a interface.

Os modelos descritos nesta seção são “abstrações daquilo que o usuário está fazendo ou pensa que está fazendo ou aquilo que alguém pensa que deveria estar fazendo quando usa um sistema interativo” [Mon84]. Em essência, esses modelos possibilitam que o projetista de interfaces satisfaça um elemento-chave do mais importante princípio do projeto de interfaces do usuário: “Conhecendo o usuário, você conhecerá as tarefas”.

11.2.2 O processo

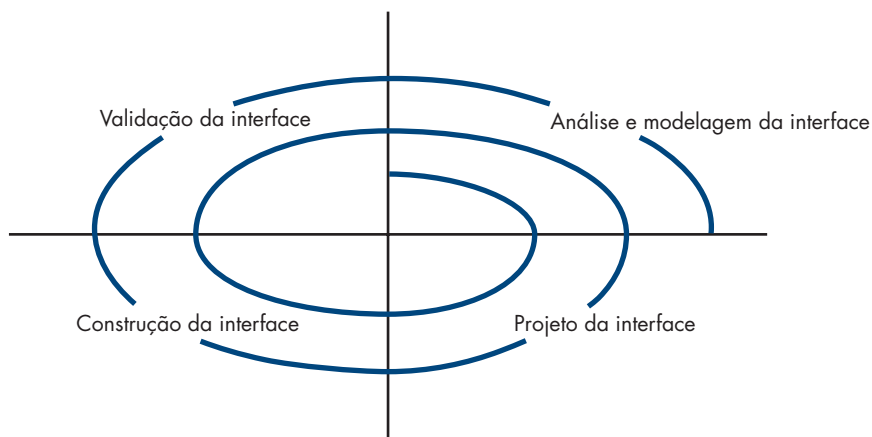
O processo de análise e projeto para interfaces do usuário é iterativo e pode ser representado usando-se um modelo espiral similar ao discutido no Capítulo 2. De acordo com a Figura 11.1, o processo de análise e projeto de interfaces do usuário começa no interior da espiral e engloba quatro atividades estruturais distintas [Man97]: (1) análise e modelagem de interfaces, (2) projeto de interfaces, (3) construção de interfaces e (4) validação de interfaces. A espiral da Figura 11.1 implica que cada uma dessas tarefas ocorrerá mais de uma vez; cada volta em torno da espiral representa a elaboração adicional dos requisitos e do projeto resultante. Na maioria

¹ Nesse contexto, *conhecimento sintático* refere-se à mecânica de interação necessária para usar a interface de forma efetiva.

² *Conhecimento semântico* refere-se ao sentido subjacente da aplicação — um entendimento das funções realizadas, a medida de entrada e saída e as metas e objetivos do sistema.

FIGURA 11.1

O processo de projeto de interface do usuário



dos casos, a atividade de construção envolve prototipagem — a única maneira prática de validar o que foi projetado.

A *análise de interfaces* focaliza o perfil dos usuários que irão interagir com o sistema. O nível de habilidades, o conhecimento da área e a receptividade geral em relação ao novo sistema são registrados e categorias de usuários são definidas. Para cada categoria de usuário, é feito o levantamento de requisitos. Em essência, trabalhamos para compreender a percepção do sistema (Seção 11.2.1) para cada classe de usuário.

Uma vez definidos os requisitos gerais, é realizada uma *análise de tarefas* mais detalhada. Aquelas tarefas que o usuário realiza para alcançar os objetivos do sistema são identificadas, descritas e elaboradas (ao longo de uma série de passagens iterativas pela espiral). A análise de tarefas é discutida de forma mais detalhada na Seção 11.3. Por fim, a análise do ambiente do usuário concentra-se no ambiente de trabalho físico. Entre as perguntas a ser feitas, temos:

- Onde estará fisicamente localizada a interface?
- O usuário estará sentado, de pé ou realizando outras tarefas não relacionadas com a interface?
- O hardware de interface leva em conta restrições de espaço, luz ou ruído?
- Existem considerações de fatores humanos especiais determinados por fatores ambientais?

As informações coletadas como parte da ação de análise são usadas para criar um modelo de análise para a interface. Usando esse modelo como base, a atividade de projeto se inicia. A meta do *projeto da interface* é definir um conjunto de objetos e ações de interface (e suas representações na tela) que permitam a um usuário realizar todas as tarefas definidas para atender todas as metas de usabilidade definidas para o sistema. O projeto de interfaces é discutido de forma mais detalhada na Seção 11.4.

A *construção da interface* em geral se inicia com a criação de um protótipo que permite que cenários de uso possam ser avaliados. À medida que o processo de projeto iterativo prossegue, um kit de ferramentas de interfaces do usuário (Seção 11.5) poderia ser usado para completar a construção da interface.

A *validação da interface* se concentra: (1) na capacidade de a interface implementar corretamente todas as tarefas de usuário, levar em conta todas as variações de tarefas, bem como atender todas os requisitos gerais dos usuários; (2) no grau de facilidade de uso e aprendizado da interface; e (3) na aceitação dos usuários da interface como uma útil ferramenta no seu trabalho.

“É melhor projetar a experiência do usuário do que retificá-la.”

Jon Meads

? O que precisamos saber sobre o ambiente ao iniciarmos o projeto de interfaces do usuário?

Conforme já citado, as atividades descritas nesta seção ocorrem iterativamente. Consequentemente, não há nenhuma necessidade de tentar especificar todos os detalhes (para modelo de análise ou de projeto) na primeira passagem. Passagens subsequentes ao longo do processo elaboram detalhes de tarefas, informações de projeto e características operacionais da interface.

11.3 ANÁLISE DE INTERFACES³

Um princípio fundamental de todos os modelos de processos de engenharia de software é o seguinte: *entender o problema antes de tentar desenvolver uma solução*. No caso do projeto de interfaces do usuário, entender o problema significa entender: (1) as pessoas (usuários finais) que irão interagir com o sistema por meio da interface, (2) as tarefas que os usuários finais devem realizar para completar seus trabalhos, (3) o conteúdo que é apresentado como parte da interface e (4) o ambiente onde essas tarefas serão conduzidas. Nas seções seguintes, examinaremos cada um dos elementos da análise de interfaces com o objetivo de estabelecer uma sólida base para as tarefas de projeto que se seguem.

11.3.1 Análise de usuários

A frase “interface do usuário” provavelmente seja a única justificativa necessária para desperdirmos algum tempo para entender o usuário antes de nos preocuparmos com as questões técnicas. Citei anteriormente que cada usuário tem uma imagem mental do software que pode ser diversa da imagem mental desenvolvida por outros usuários. Além do mais, a imagem mental do usuário pode ser muito diferente do modelo de projeto do engenheiro de software. A única maneira para se fazer com que a imagem mental e o modelo de projeto convirjam é tentar entender os próprios usuários, bem como as pessoas que usam o sistema. Informações de uma ampla gama de fontes podem ser usadas para concretizar isso:

 **Como saber o que o usuário quer da interface do usuário?**

Entrevistas com os usuários. Na abordagem mais direta, membros da equipe de software se encontram com os usuários finais para entender melhor suas necessidades, motivações, cultura de trabalho e uma miríade de outras questões. Isso pode ser conseguido em reuniões um-a-um ou por grupos de sondagem.

Informações do pessoal de vendas. O pessoal de vendas se encontra regularmente com usuários e pode reunir informações que ajudarão a equipe de software a classificar os usuários e compreender melhor seus requisitos.

Informações do pessoal de marketing. A análise de mercado pode ser inestimável na definição de segmentos de mercado e no entendimento de como cada segmento poderia usar o software de modos sutilmente diferentes.

Informações do pessoal de suporte. A equipe de suporte conversa diariamente com os usuários. Eles são, provavelmente, a fonte mais provável de informações sobre o que funciona e o que não funciona, o que os usuários gostam e o que não gostam, quais recursos geram questionamentos e quais são fáceis de ser usados.

O conjunto de perguntas a seguir (adaptado de [Hac98]) irá ajudá-lo a entender melhor os usuários de um sistema:

- Os usuários são profissionais treinados, técnicos, do setor administrativo ou pessoal de fábrica?
- Que nível de educação formal o usuário médio possui?
- Os usuários são capazes de aprender por meio de material escrito ou expressaram seu desejo por um treinamento em sala de aula?



Acima de tudo, invista tempo conversando com os usuários de fato, mas seja cauteloso. Uma opinião firme não significa necessariamente que a maioria dos usuários irá concordar.

PONTO-CHAVE

Como colocar-se a par da demografia e características dos usuários finais?

³ É razoável argumentar que essa seção deveria ser colocada no Capítulo 5, 6 ou 7, já que questões de análise de requisitos são discutidas lá. Ela foi inserida aqui porque a análise e o projeto de interfaces estão intimamente ligados entre si e a fronteira entre os dois em geral é confusa.

- Os usuários são digitadores experientes ou têm fobia a teclados?
- Qual a faixa etária da comunidade de usuários?
- Os usuários serão representados predominantemente por um gênero?
- Como os usuários são recompensados pelo trabalho realizado?
- Os usuários trabalham em expediente normal ou ficam até que o trabalho seja completado?
- O software deverá ser parte integrante dos usuários de trabalho ou será usado apenas esporadicamente?
- Qual o principal idioma falado pelos usuários?
- Quais as consequências se um usuário cometer um erro ao usar o sistema?
- Os usuários são especialistas no assunto tratado pelo sistema?
- Os usuários querem saber sobre a tecnologia que se encontra por trás da interface?

Assim que essas perguntas forem respondidas, saberemos quem são os usuários finais, o que provavelmente irá motivá-los, como poderão ser agrupados em diferentes classes ou perfis, quais são seus modelos mentais do sistema e como uma interface deve ser caracterizada para atender suas necessidades.

11.3.2 Análise e modelagem de tarefas

O objetivo da análise de tarefas é responder às seguintes questões:

- Que trabalho o usuário irá realizar em circunstâncias específicas?
- Quais tarefas e subtarefas serão realizadas à medida que o usuário desenvolve seu trabalho?
- Quais os objetos de domínio de problema específicos que o usuário irá manipular à medida que o trabalho é desenvolvido?
- Qual a sequência de tarefas — o fluxo de trabalho?
- Qual a hierarquia das tarefas?

Para responder a essas questões, deve-se fazer uso das técnicas discutidas anteriormente neste livro, mas neste caso, elas são aplicadas à interface do usuário.

Casos de uso. Em capítulos anteriores vimos que um caso de uso descreve a maneira pela qual um ator (no contexto do projeto de interfaces com o usuário, um ator é sempre uma pessoa) interage com um sistema. Quando usado como parte da análise de tarefas, o caso de uso é desenvolvido para mostrar como um usuário final realiza alguma tarefa relacionada com algum trabalho específico. Na maioria das oportunidades, o caso de uso é redigido em estilo informal (um parágrafo simples) na primeira pessoa. Suponhamos, por exemplo, que uma pequena empresa de software queira construir um sistema de projeto apoiado por computador explicitamente para arquitetos de interiores. Para se ter uma ideia de como eles realizam seus trabalhos, solicita-se a arquitetos de interiores que descrevam uma função de projeto específica. Ao ser indagado “Como você decide onde colocar o mobiliário em uma sala?”, um arquiteto de interiores escreveu o seguinte caso de uso informal:

Começo esboçando a planta baixa da sala, as dimensões e a localização das janelas e portas. Preocupo-me muito com a iluminação do ambiente, com a vista das janelas (caso seja bonita, quero que sobressaia), com o comprimento total livre de uma parede, com o fluxo de movimento pelo ambiente. Em seguida, pesquiso a lista de mobiliário que eu e meu cliente escolhemos — mesas, cadeiras, sofás, armários, a lista de acessórios — lâmpadas, tapetes, pinturas, escultura, plantas, peças menores e minhas anotações sobre quaisquer desejos que meu cliente tenha a colocar. Em seguida, desenho cada item das minhas listas usando um gabarito colocado em escala na planta. Nomeio cada item que desenho e uso um lápis, pois sempre acabo mudando as coisas. Considero uma série de posições alternativas e opto por aquela que mais me agrada. Em seguida, crio um *rendering* (uma figura 3-D) do ambiente, para que meu cliente tenha uma sensação de como ele ficará.

PONTO-CHAVE

A meta do usuário é realizar uma ou mais tarefas via interface do usuário. Para tanto, a interface deve fornecer mecanismos que permitam ao usuário atingir sua meta.

Esse caso de uso dá uma descrição básica de uma importante tarefa para o sistema de projeto com auxílio de computador. Por meio dele, podemos extrair tarefas, objetos e o fluxo geral da interação. Além disso, outras características do sistema que poderiam agradar o arquiteto de interiores também poderiam ser concebidas. Por exemplo, poderíamos tirar uma foto digital da vista de cada janela do ambiente. Quando o ambiente passar pelo processo de *rendering*, a vista externa real poderia ser representada através de cada janela.



A elaboração de tarefas é bem útil, porém, também pode ser perigosa. Não pressuponha, apenas por ter elaborado uma tarefa, que não exista uma outra maneira de realizá-la e que essa outra maneira será tentada quando a interface do usuário for implementada.

Elaboração de tarefas. No Capítulo 8, discutimos a elaboração gradual (também denominada decomposição funcional ou refinamento gradual) como um mecanismo para refinar as tarefas de processamento necessárias para o software realizar alguma função desejada.

A análise de tarefas para projeto de interfaces usa uma abordagem de refinamento para auxiliar a entender as atividades humanas que uma interface do usuário deve atender. A análise de tarefas pode ser aplicada de duas formas. Conforme já citado, em geral, é usado um sistema computacional interativo para substituir uma atividade manual ou semimanual. Para compreendermos as tarefas que devem ser realizadas para atingir a meta da atividade, temos de entender as tarefas que as pessoas realizam atualmente (ao usar um método manual) e, em seguida, mapeá-las em um conjunto de tarefas similar (mas não necessariamente idêntico) implementadas no contexto de uma interface do usuário. Como alternativa, podemos estudar uma especificação existente para uma solução baseada em computador e obter um conjunto de tarefas de usuário que irá atender o modelo de usuário, o modelo de projeto e a percepção do sistema.

CASA SEGURA



Casos de uso para o projeto de interfaces do usuário

Cena: Sala do Vinod, enquanto prossegue o projeto de interface do usuário.

Atores: Vinod e Jamie, membros da equipe de engenharia de software do CasaSegura.

Conversa:

Jamie: Fiz com que nosso contato de marketing redigisse um caso de uso para a interface de vigilância.

Vinod: Do ponto de vista de quem?

Jamie: Do proprietário do imóvel, quem mais poderia ser?

Vinod: Há também o papel do administrador do sistema, mesmo que o próprio proprietário esteja desempenhando a função, é um ponto de vista diferente. O “administrador” ativa o sistema, configura as coisas, faz o layout da planta, posiciona as câmeras...

Jamie: Em suma, desempenha o papel do proprietário quando ele quiser assistir ao vídeo.

Vinod: Está bem. Esse é um dos principais comportamentos da interface da função de vigilância. Mas também teremos de examinar o comportamento da administração do sistema.

Jamie (irritado): Você está certo.

[Jamie sai em busca da pessoa de marketing. Ele retorna algumas horas mais tarde.]

Jamie: Tive sorte, encontrei-a e trabalhamos juntos no caso de uso do administrador. Basicamente, iremos definir “administra-

ção” como uma função que se aplica a todas as demais funções do CasaSegura. Eis a que conclusão chegamos.

[Jamie mostra o caso de uso informal a Vinod.]

Caso de uso informal: Quero ser capaz de estabelecer ou editar o layout do sistema a qualquer momento. Ao configurar o sistema, seleciono uma função administrativa. Ela me pergunta se quero fazer uma nova configuração ou editar uma já existente. Caso opte por uma nova configuração, o sistema exibe uma tela de desenho que me permitirá desenhar a planta em uma grade de pontos. Existirão ícones para as paredes, janelas e portas para facilitar o desenho. Tenho simplesmente que “esticar” os ícones até atingirem os comprimentos apropriados. O sistema irá mostrar os comprimentos em pés ou metros (posso escolher o sistema de medidas). Posso escolher de uma biblioteca de sensores e câmeras e colocá-los na planta. Tenho que dar nome a cada um deles ou deixar que o sistema faça automaticamente. Posso estabelecer ajustes para os sensores e câmeras por meio de menus apropriados. Caso opte por editar, poderei movimentar os sensores ou as câmeras, acrescentar novas(os) ou eliminar algum(a) existente, editar a planta, bem como os ajustes de configuração para as câmeras e sensores. Em cada um dos casos, espero que o sistema realize testes de consistência e me ajude a não cometer erros.

Vinod (após ler o cenário): Certo, provavelmente existem alguns úteis padrões de projeto [Capítulo 12] ou componentes reutilizáveis para GUIs para programas de desenho. Posso apostar 50 paus que conseguimos implementar parte ou a maior parte da interface de administrador usando-os.

Jamie: De acordo. Verificarei isso.

Independentemente da abordagem geral para a análise de tarefas, temos de, primeiramente, definir e classificar as tarefas. Já citei que uma das abordagens é a elaboração gradual. Reconsideremos, por exemplo, o sistema de projeto auxiliado por computador para arquitetos de interiores, discutido anteriormente. Observando um arquiteto de interiores em trabalho, percebemos que o projeto do interior compreende uma série de atividades principais: layout do mobiliário (note o caso de uso discutido anteriormente), escolha de tecidos e materiais, escolha de papéis de parede e acabamentos para janelas, apresentação (para o cliente), custo e compras. Cada uma das tarefas principais pode ser elaborada em subtarefas. Por exemplo, usando informações contidas no caso de uso, o layout de mobiliário pode ser refinado nas seguintes tarefas: (1) desenhar uma planta nas dimensões do ambiente, (2) colocar as janelas e as portas nos locais apropriados, (3a) usar gabaritos de mobiliário para desenhar contornos de mobiliário em escala na planta, (3b) usar gabaritos de acessórios para desenhar acessórios em escala na planta, (4) movimentar contornos de mobiliário e de acessórios para obter o melhor posicionamento, (5) identificar todos os contornos de mobiliário e acessórios, (6) indicar as dimensões para mostrar as posições e (7) desenhar uma vista em perspectiva para o cliente. Poderia ser usada uma abordagem similar para cada uma das principais tarefas.

As subtarefas 1 a 7 podem ser refinadas ainda mais. As subtarefas 1 a 6 serão realizadas por meio da manipulação das informações e realização de ações em uma interface do usuário. Por outro lado, a subtarefa 7 pode ser realizada automaticamente no software e resultará pouca interação⁴ direta com o usuário. O modelo de projeto da interface deve considerar cada uma das tarefas de maneira consistente com o modelo de usuário (o perfil de um “típico” arquiteto de interiores) e a percepção do sistema (o que o arquiteto de interiores espera de um sistema automatizado).

Elaboração de objetos. Em vez de nos concentrarmos nas tarefas que um usuário tem de realizar, podemos examinar o caso de uso e outras informações obtidas do usuário e extrair os objetos físicos utilizados pelo arquiteto de interiores. Tais objetos podem ser classificados em classes. São definidos os atributos de cada classe e uma avaliação das ações aplicadas a cada objeto fornece uma lista de operações. Por exemplo, o gabarito de mobiliário talvez pudesse ser traduzido em uma classe chamada **Mobiliário** com atributos que poderiam incluir **tamanho, forma, posição** e outros. O arquiteto de interiores *selecionaria* o objeto da classe **Mobiliário**, o *moveria* para uma posição na planta (um outro objeto nesse contexto), *desenharia* o contorno da móvel e assim por diante. As tarefas *selecionar*, *mover* e *desenhar* são operações. O modelo de análise da interface do usuário não forneceria uma implementação literal para cada uma dessas operações. Entretanto, à medida que o projeto é elaborado, os detalhes das operações são definidos.

Análise do fluxo de trabalho. Quando uma série de usuários, desempenhando diferentes papéis, faz uso de uma interface do usuário, algumas vezes se faz necessário ir além da análise de tarefas e da elaboração dos objetos e aplicar a *análise do fluxo de trabalho*. Essa técnica permite que entendamos como um processo de trabalho é completado quando várias pessoas (e papéis) estão envolvidas. Consideremos uma empresa que pretenda automatizar completamente o processo de prescrição e entrega de medicamentos. Todo o processo⁵ irá girar em torno de uma aplicação baseada na Web que pode ser acessada por médicos (ou seus assistentes), farmacêuticos e pacientes. O fluxo de trabalho pode ser representado efetivamente por um diagrama de raias UML (uma variação do diagrama de atividades).

Consideramos apenas uma pequena parte do processo de trabalho: a situação que ocorre quando um paciente solicita uma revalidação da receita. A Figura 11.2 apresenta um diagrama de raias que indica as tarefas e decisões para cada um dos três papéis citados anteriormente.



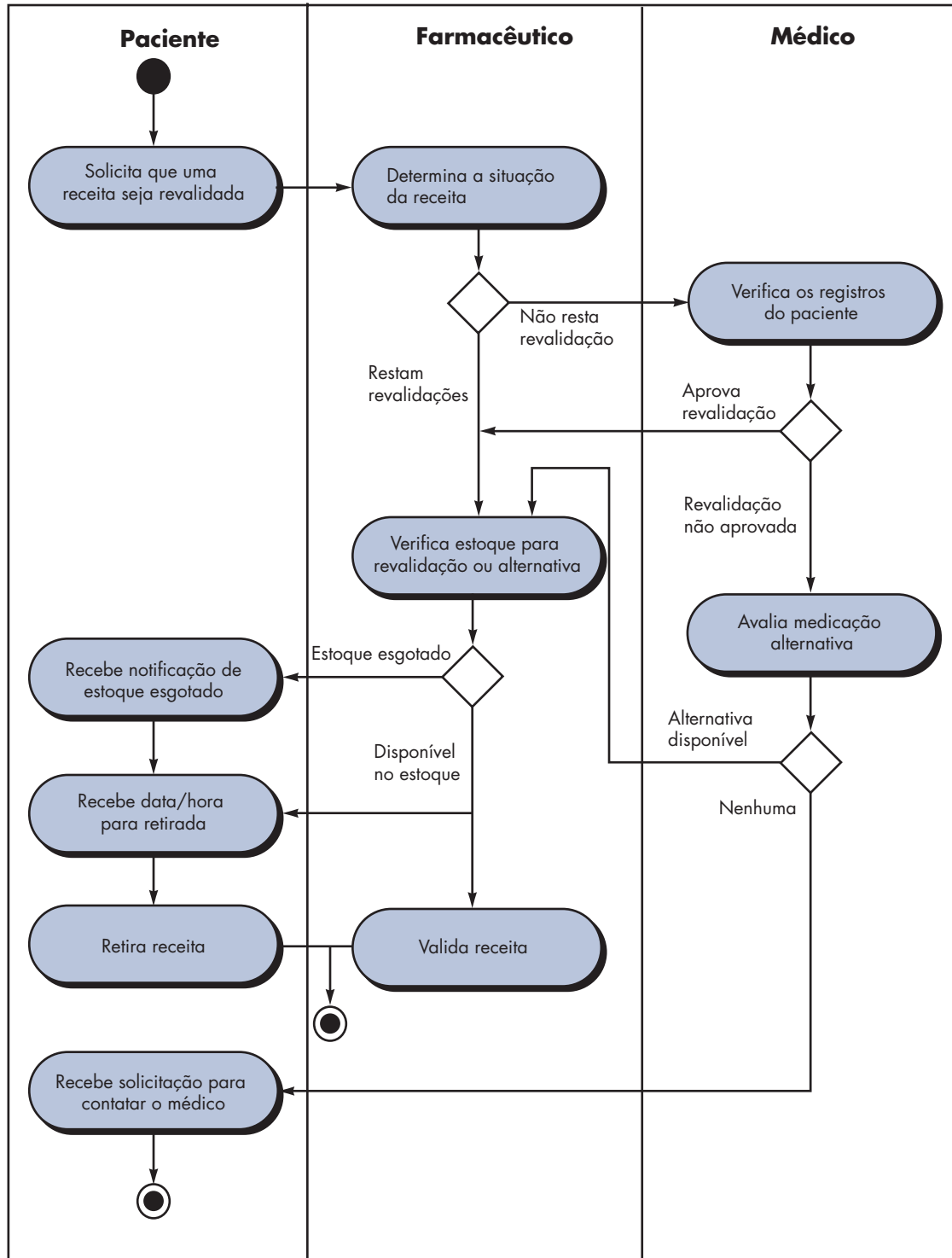
Embora a elaboração de objetos seja útil, não deve ser usada como um método isolado. A opinião do usuário tem de ser considerada durante a análise de tarefas.

⁴ Entretanto, talvez esse não seja o caso. O arquiteto de interiores talvez queira especificar a perspectiva a ser desenhada, a escala, o uso de cores e outras informações. O caso de uso relacionado ao desenho de vistas em perspectiva com efeito de rendering forneceria as informações necessárias para lidar com essa tarefa.

⁵ Esse exemplo foi adaptado de [Hac98].

FIGURA 11.2

Diagrama de raias para a função de revalidação de receita



As informações talvez sejam levantadas via entrevista ou por meio de casos de uso escritos por cada ator. Independentemente disso, o fluxo de eventos (mostrado na figura) nos permite reconhecer uma série de características-chave da interface:

“É muito melhor adaptar a tecnologia ao usuário do que forçá-lo a se adaptar à tecnologia.”

Larry Marine

1. Cada usuário implementa tarefas distintas via interface; consequentemente, o aspecto da interface projetada para o paciente será diferente do definido para os farmacêuticos ou médicos.
2. O projeto da interface para os farmacêuticos e médicos deve contemplar acesso e exibição de informações de fontes de informação secundárias (por exemplo, acesso ao estoque para o farmacêutico e acesso a informações sobre medicamentos alternativos para o médico).
3. Muitas das atividades citadas no diagrama de raia podem ser elaboradas ainda mais com o uso de análise de tarefas e/ou elaboração de objetos (por exemplo, *Preenche receita* poderia implicar uma entrega via correio, uma ida à farmácia ou a um centro de distribuição de medicamentos especial).

Representação hierárquica. Um processo de elaboração ocorre quando se começa a analisar a interface. Uma vez estabelecido o fluxo de trabalho, pode ser definida uma hierarquia de tarefas para cada tipo de usuário. A hierarquia é obtida por meio de uma elaboração gradual de cada tarefa identificada para o usuário. Consideremos, por exemplo, a seguinte tarefa de usuário e sub-hierarquia de tarefas.

Tarefa de usuário: *Solicita que uma receita seja revalidada*

- *Fornecer informações de identificação.*
 - *Especificar nome.*
 - *Especificar identificação do usuário.*
 - *Especificar PIN e senha.*
- *Especificar número da receita.*
- *Especificar se é exigida a data de revalidação.*

Para completar a tarefa, são definidas três subtarefas. Uma das subtarefas, *fornecer informações de identificação*, é elaborada ainda mais em três subsubtarefas adicionais.

11.3.3 Análise do conteúdo exibido

As tarefas de usuário identificadas na Seção 11.3.2 levam à apresentação de uma série de tipos diferentes de conteúdo. Para aplicações modernas, o conteúdo exibido pode variar de relatórios de texto (por exemplo, uma planilha), visualização gráfica (por exemplo, um histograma, um modelo 3-D, a foto de uma pessoa) ou informações especializadas (por exemplo, arquivos de áudio ou vídeo). As técnicas de modelagem de análise discutidas nos Capítulos 6 e 7 identificam os objetos de dados de saída produzidos por uma aplicação. Tais objetos de dados poderiam ser: (1) gerados por componentes (não relacionados com a interface) em outras partes de uma aplicação, (2) obtidos de dados armazenados em um banco de dados acessível para a aplicação ou (3) transmitidos de sistemas externos à aplicação em questão.

Durante essa etapa de análise de interface, são considerados o formato e a estética do conteúdo (como ele é exibido pela interface). Entre as perguntas feitas e respondidas, temos:

- Os diferentes tipos de dados são alocados em posições geográficas consistentes na tela (por exemplo, fotos sempre apareceriam no canto superior direito)?
- O usuário pode personalizar a localização do conteúdo na tela?
- Foi atribuída identificação apropriada na tela para todos os conteúdos?
- Se for preciso apresentar um relatório grande, como seria subdividido para facilitar sua compreensão?
- Haverá mecanismos disponíveis para ir diretamente a informações resumidas em conjuntos de dados volumosos?

 Como determinar o formato e a estética de conteúdo exibido como parte da interface do usuário?

- A saída gráfica será apresentada em escala para caber nos limites do dispositivo de exibição utilizado?
- Como serão usadas cores para melhorar o entendimento?
- Como serão apresentadas ao usuário mensagens de erro e alertas?

As respostas a essas (e outras) questões nos ajudarão a estabelecer os requisitos para apresentação de conteúdo.

11.3.4 Análise do ambiente de trabalho

Hackos e Redish [Hac98] discutem a importância da análise do ambiente de trabalho ao afirmarem:

As pessoas não realizam seus trabalhos de forma isolada. São influenciadas pela atividade em torno delas, pelas características físicas do local de trabalho, pelo tipo de equipamento utilizado e pelas relações de trabalho que têm com outras pessoas. Se os produtos projetados não se ajustarem ao ambiente, serão difíceis ou frustrantes de ser usados.

Em algumas aplicações, a interface do usuário para um sistema baseado em computador é colocada em uma “posição que facilita o usuário” (por exemplo, iluminação apropriada, altura adequada da tela, fácil acesso ao teclado), porém em outras (por exemplo, no chão de fábrica ou no *cockpit* de um avião), talvez a iluminação não seja tão adequada, o ruído pode ser um fator importante, um teclado ou mouse talvez não sejam uma opção, o posicionamento da tela talvez seja abaixo do ideal. O projetista de interfaces talvez esteja restrito por fatores que reduzam a facilidade de uso.

Além dos fatores ambientais físicos, a cultura do local de trabalho também entra em cena. A interação do sistema será medida de alguma maneira (por exemplo, tempo por transação ou precisão de uma transação)? Duas ou mais pessoas terão de compartilhar informações antes de uma opinião poder ser fornecida? Como será oferecido suporte aos usuários do sistema? Essas e muitas outras questões relacionadas devem ser respondidas antes de o projeto de interface se iniciar.

11.4 ETAPAS NO PROJETO DE INTERFACES

“O projeto interativo [é] uma mescla perfeita de arte gráfica, tecnologia e psicologia.”

Brad Wieners

Assim que a análise de interface tiver sido completada, todas as tarefas (ou objetos e ações) requeridas pelo usuário final foram identificadas de forma detalhada e a atividade de projeto de interface se inicia. O projeto de interfaces, assim como todo projeto de engenharia de software, é um processo iterativo. Cada etapa de um projeto de interface do usuário ocorre uma série de vezes, elaborando e refinando informações desenvolvidas na etapa anterior.

Embora tenham sido propostos diversos modelos diferentes para projeto de interfaces do usuário (por exemplo, [Nor86], [Nie00]), todos sugerem alguma combinação das seguintes etapas:

1. Usar informações desenvolvidas durante a análise de interfaces (Seção 11.3), definir objetos e ações (operações) de interface.
2. Definir eventos (ações de usuário) que provocarão a mudança de estado de uma interface do usuário. Modelar esse comportamento.
3. Representar cada estado da interface como realmente aparecerá para o usuário final.
4. Indicar como o usuário interpreta o estado do sistema com base em informações fornecidas através da interface.

Em alguns casos, podemos começar com esboços de cada estado de interface (ou seja, qual o aspecto da interface do usuário sob diversas circunstâncias) e então trabalhar no sentido inverso para definir objetos, ações e outras importantes informações de projeto. Independentemente da sequência de tarefas de projeto, devemos: (1) sempre seguir as regras de ouro discutidas na

Seção 11.1, (2) modelar como a interface será implementada e (3) considerar o ambiente (por exemplo, tecnologia de exibição, sistema operacional, ferramentas de desenvolvimento) a ser usado.

11.4.1 Aplicação das etapas para projeto de interfaces

A definição dos objetos da interface e as ações aplicadas a eles é uma importante etapa no projeto. Para concretizar isso, cenários de usuário são analisados sintaticamente quase da mesma forma descrita no Capítulo 6. Ou seja, é escrito um caso de uso. Os substantivos (objetos) e os verbos (ações) são isolados para criar uma lista de objetos e ações.

Assim que os objetos e ações tiverem sido definidos e elaborados iterativamente, são classificados por tipo. Identificam-se objetos de destino, origem e aplicação. Um *objeto de origem* (por exemplo, um ícone de relatório) é arrastado e solto sobre um *objeto de destino* (por exemplo, um ícone de impressora). A implicação dessa ação é criar um relatório impresso. Um *objeto de aplicação* representa dados específicos à aplicação que não são diretamente manipulados como parte da interação de tela. Por exemplo, uma lista de endereços é usada para armazenar nomes para uma postagem. A própria lista poderia ser classificada, combinada ou filtrada (ações baseadas em menus), mas ela não é arrastada e solta através de interação com o usuário.

Quando estiver satisfeito com todas as ações e objetos importantes definidos (para uma iteração de projeto), realize o layout da tela. Assim como outras atividades do projeto de interfaces, o layout da tela é um processo interativo no qual são realizados o projeto gráfico e o posicionamento dos ícones, a definição de texto de tela descritivo, a especificação e a colocação de títulos para as janelas, bem como a definição de itens de menu principais e secundários. Se for apropriada para a aplicação uma metáfora do mundo real, esta é especificada neste momento e o layout é organizado para complementar tal metáfora.

Para darmos um breve exemplo das etapas de projeto citadas anteriormente, consideremos um cenário de usuário para o sistema *CasaSegura* (discutido em capítulos anteriores). Segue um caso de uso preliminar (redigido pelo proprietário do imóvel) para a interface:

Caso de uso preliminar: Quero ter acesso ao meu sistema *CasaSegura* de qualquer ponto remoto via Internet. Por meio de um navegador instalado em meu notebook (enquanto estou no trabalho ou viajando), posso determinar o estado do sistema de alarme, armar ou desarmá-lo, reconfigurar zonas de segurança e ver diferentes ambientes da casa via câmeras de vídeo pré-instaladas.

Para acessar o *CasaSegura* de um ponto remoto, forneço um identificador de usuário e uma senha. Estes definem níveis de acesso (por exemplo, nem todo usuário poderá reconfigurar o sistema) e dão segurança. Uma vez validados, posso verificar o estado do sistema e alterá-lo armando ou desarmando o *CasaSegura*. Posso reconfigurar o sistema exibindo uma planta da casa, vendo cada um dos sensores de segurança, exibindo cada zona configurada atualmente e modificando as zonas conforme necessário. Posso ver o interior da casa via câmeras de vídeo estrategicamente colocadas. Posso deslocar e ampliar a imagem de cada câmera para ter visões diferentes do seu interior.

Tomando como base esse caso de uso, são identificados as seguintes tarefas, objetos e dados do proprietário do imóvel:

- *Acessa* o sistema *CasaSegura*.
- *Introduz* um **ID** e **senha** para permitir acesso remoto.
- *Verifica* **estado do sistema**.
- *Arma* ou *desarma* o sistema *CasaSegura*.
- *Exibe* **planta** e **localização dos sensores**.
- *Exibe* **zonas** na planta.
- *Altera* **zonas** na planta.
- *Exibe* **localização das câmeras de vídeo** na planta.
- *Seleciona* **câmera de vídeo** para visualização.
- *Visualiza* **imagens de vídeo** (quatro quadros por segundo).
- *Desloca* ou *amplia* o **foco da câmera de vídeo**.



Embora as ferramentas automatizadas possam ser úteis no desenvolvimento de protótipos de layout, algumas vezes basta um lápis e papel.

Os objetos (em negrito) e as ações (em *itálico*) são extraídos da lista de tarefas do proprietário do imóvel. A maioria dos objetos citados são objetos de aplicação. Entretanto, **localização das câmeras de vídeo** (um objeto de origem) é arrastado e solto sobre **câmera de vídeo** (um objeto de destino) para criar uma **imagem de vídeo** (uma janela com exibição de vídeo).

É criado um esboço preliminar do layout da tela para monitoramento de vídeo (Figura 11.3).⁶ Para chamar a imagem de vídeo, é selecionado um ícone de localização de câmera de vídeo, C, localizado na planta exibida na janela de monitoramento. Nesse caso, a localização da câmera na sala (SE) é então arrastada e solta sobre o ícone de câmera de vídeo no canto superior esquerdo da tela. Surge a janela de imagem de vídeo, mostrando vídeo *streaming* da câmera localizada em SE. Os controles deslizantes para controle de ampliação e deslocamento são usados para controlar a ampliação e a direção da imagem de vídeo. Para selecionar uma vista de outra câmera, o usuário apenas arrasta e solta um ícone de localização da câmera diferente sobre o ícone de câmera no canto superior esquerdo da tela.

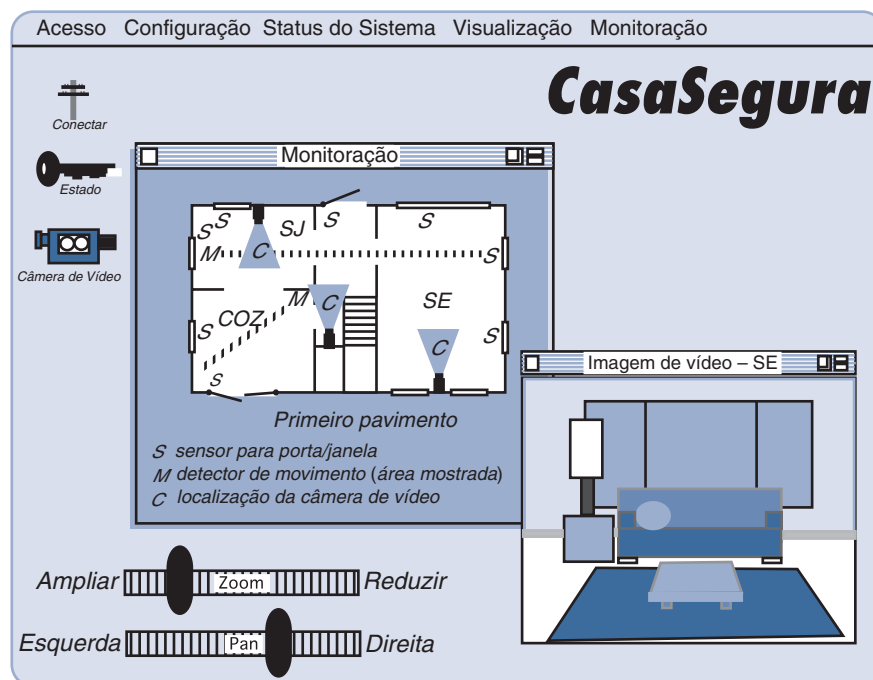
O esboço do layout teria de ser complementado com uma expansão de cada item de menu contido na barra de menus, indicando que ações estão disponíveis para o modo (estado) de monitoramento de vídeo. Um conjunto completo de esboços para cada tarefa do proprietário do imóvel citada no cenário de usuário seria criado durante o projeto de interfaces.

11.4.2 Padrões de projeto de interfaces do usuário

As interfaces gráficas do usuário tornaram-se tão comuns que surgiu uma ampla gama de padrões de projeto de interfaces. Conforme citado anteriormente neste livro, um padrão de projeto é uma abstração que prescreve uma solução de projeto para um problema de projeto específico e bem delimitado.

FIGURA 11.3

Layout preliminar da tela



⁶ Observe que este difere ligeiramente da implementação desses recursos em capítulos anteriores. Isso poderia ser considerado um projeto preliminar e representa uma alternativa que possa vir a ser considerada.

WebRef

Foi proposta uma ampla gama de padrões de projeto para interfaces do usuário. Para encontrar uma grande variedade de sites com esse tipo de padrões, visite www.hcipatterns.org.

Como exemplo de problema comumente encontrado para projeto de interfaces, consideremos a situação em que um usuário tem de introduzir uma ou mais datas, às vezes com meses de antecedência. Existem várias soluções possíveis para esse problema simples e uma série de padrões diferentes que poderiam ser propostos. Laakso [Laa00] sugere um padrão denominado **FaixaDeCalendário**, que produz um calendário contínuo e que rola onde a data atual é destacada e datas futuras podem ser selecionadas indicando-as no calendário. A metáfora de calendário é bem conhecida de todo usuário e oferece um mecanismo efetivo para colocar uma data futura no contexto.

Ao longo da última década foram propostos vários padrões de projeto de interfaces. Uma discussão mais detalhada sobre padrões de projeto de interfaces do usuário é apresentada no Capítulo 12. Além disso, Erickson [Eri08] indica links para vários conjuntos baseados na Web.

11.4.3 Questões de projeto

À medida que o projeto de uma interface do usuário evolui, quatro questões de projeto comuns quase sempre vêm à tona: tempo de resposta do sistema, recursos de ajuda ao usuário, informações de tratamento de erros e atribuição de nomes a comandos. Infelizmente, muitos projetistas não tratam desses problemas até um ponto relativamente avançado do processo de projeto (algumas vezes a primeira vaga ideia de um problema não ocorre até que um protótipo operacional esteja disponível). Em geral, decorrem iterações desnecessárias, atrasos de projeto e frustração por parte do usuário final. É muito melhor estabelecer cada um desses como um problema de projeto a ser considerado no início do projeto de software, quando as mudanças são fáceis e custam pouco.

Tempo de resposta. O tempo de resposta do sistema é a principal queixa para várias aplicações interativas. Em geral, o tempo de resposta do sistema vai do ponto no qual o usuário realiza alguma ação de controle (por exemplo, aciona a tecla <Enter> ou clica o mouse) até que o software responda com saída ou ação desejadas.

O tempo de resposta do sistema apresenta duas importantes características: duração e variabilidade. Se a resposta do sistema for muito longa, frustração e estresse por parte do usuário serão inevitáveis. A *variabilidade* refere-se ao desvio do tempo de resposta médio e, em vários aspectos, é a característica de tempo de resposta mais importante. Baixa variabilidade permite ao usuário estabelecer um ritmo de interação, mesmo que o tempo de resposta seja relativamente longo. Por exemplo, uma resposta de 1 segundo a um comando normalmente é preferível do que uma resposta que varia de 0,1 a 2,5 segundos. Quando a variabilidade é significativa, o usuário sempre se desequilibra, sempre conjecturando se algo “diferente” ocorreu ou não nos bastidores.

Recursos de ajuda. Quase todo usuário de um sistema computacional interativo requer ajuda de vez em quando. Em alguns casos, uma simples questão dirigida a um colega com bons conhecimentos pode resolver a questão. Em outros, uma pesquisa detalhada em um conjunto de vários volumes de “manuais de usuário” talvez seja a única alternativa. Na maioria das vezes, entretanto, os aplicativos modernos fornecem recursos de ajuda on-line que permitem a um usuário obter a resposta para determinada questão ou resolver um problema sem ter de abandonar a interface.

Uma série de problemas de projeto [Rub88] tem de ser tratada quando se considera um recurso de ajuda de sistema:

- A ajuda estará disponível para todas as funções do sistema e a qualquer momento durante a interação com o sistema? Há a possibilidade de incluir ajuda apenas para um subconjunto de todas as funções e ações ou ajuda para todas as funções.
- Como o usuário solicitará ajuda? Entre as opções temos um menu de ajuda, uma tecla de função especial ou um comando HELP.
- Como será representada a ajuda? Entre as opções temos uma janela separada, uma referência a um documento impresso (não ideal) ou uma sugestão de uma ou duas linhas produzida em uma localização fixa da tela.

“Um erro comum que as pessoas cometem ao tentar projetar algo completamente infalível é subestimar a criatividade de verdadeiros tolos.”

Douglas Adams

- Como o usuário irá retornar à interação normal? Entre as opções temos um botão de retorno exibido na tela, uma tecla de função ou uma sequência de controle.
- Como as informações do sistema de ajuda serão estruturadas? Entre as opções temos uma estrutura “plana” em que todas as informações são acessadas por meio de uma palavra-chave, uma hierarquia de informações em camadas que dá cada vez mais detalhes à medida que o usuário vai avançando nessa estrutura ou o emprego de hipertexto.

“A interface do inferno: ‘Para corrigir esse erro e prosseguir, introduza qualquer número primo de 11 dígitos ...’”

Autor desconhecido

Tratamento de erros. As mensagens de erros e alertas são “más notícias” fornecidas aos usuários de sistemas interativos quando algo saiu mal. No pior caso, as mensagens de erro e alertas transmitem informações inúteis ou confusas e servem apenas para aumentar a frustração do usuário. Há poucos usuários de computador que ainda não depararam com um erro do tipo: *‘A aplicação XXX foi forçada a sair; pois um erro do tipo 1023 foi encontrado’*. Em algum lugar deve existir uma explicação para o erro 1023; caso contrário, para que aqueles que projetaram o sistema colocaram tal identificação? Contudo, a mensagem de erro não dá nenhuma indicação real do que deu errado ou onde procurar informações adicionais. Uma mensagem de erro apresentada dessa maneira não faz nada para diminuir a ansiedade do usuário ou ajudá-lo a corrigir o problema.

Em geral, toda mensagem de erro ou alerta produzida por um sistema interativo deve apresentar as seguintes características:

- A mensagem deve descrever o problema em um jargão que o usuário consiga entender.
- A mensagem deve fornecer conselhos construtivos para recuperação do erro.
- A mensagem deve indicar quaisquer consequências negativas do erro (por exemplo, arquivos de dados provavelmente corrompidos), de modo que o usuário possa fazer uma verificação para garantir que não tenham ocorrido (ou corrigi-las, caso tenham ocorrido).
- A mensagem deve ser acompanhada por algum sinal audível ou visual. Ou seja, poderia ser gerado um bipe acompanhando a exibição da mensagem ou a mensagem poderia piscar momentaneamente ou ser exibida em uma cor facilmente reconhecível como “cor de erro”.
- A mensagem deve ser “não sentenciosa”. Isto é, os termos jamais devem colocar a culpa no usuário.

Pelo fato de ninguém gostar de más notícias, poucos usuários apreciarão uma mensagem de erro independentemente da qualidade de sua formulação. Porém, uma filosofia de mensagens de erro efetiva pode contribuir muito para aumentar a qualidade de um sistema interativo e reduzirá significativamente a frustração dos usuários quando da ocorrência de problemas.

Atribuição de nomes a comandos e menus. O comando digitado já foi o modo mais comum de interação entre o usuário e um software de sistema e era usado comumente para aplicações de todo tipo. Hoje em dia, o uso de interfaces orientadas a janelas e apontar e clicar reduziram a dependência de comandos digitados, mas alguns usuários com maior conhecimento ainda preferem um modo de interação orientado a comandos. Surge uma série de problemas de projeto quando são fornecidos comandos digitados ou identificadores de menus como modo de interação:

- Toda opção de menu terá um comando correspondente?
- Qual a forma a ser assumida pelos comandos? As opções incluem uma sequência de controle (por exemplo, alt-P), teclas de função ou uma palavra digitada.
- Qual será o grau de dificuldade para aprender e lembrar-se dos comandos? O que pode ser feito se um comando for esquecido?
- Os comandos podem ser personalizados ou abreviados pelo usuário?
- Os identificadores de menus são autoexplicativos no contexto da interface?
- Os submenus são consistentes com a função sugerida por um item de menu principal?

Quais características uma “boa” mensagem de erro deve apresentar?

Conforme citado neste capítulo, as convenções para o uso de comandos devem ser estabelecidas para todas as aplicações. É confuso e normalmente sujeito a erros um usuário digitar alt-D quando um objeto gráfico tem de ser duplicado em determinada aplicação e alt-D quando um objeto gráfico tem de ser deletado em outra aplicação. A probabilidade para ocorrência de erro é óbvia.

WebRef

Diretrizes para o desenvolvimento de software acessível podem ser encontradas em www3.ibm.com/able/guidelines/software/accesssoftware.html.

Acessibilidade às aplicações. À medida que as aplicações de computador tornam-se comuns, os engenheiros de software devem garantir que o projeto de interfaces englobe mecanismos que permitam fácil acesso para aqueles com necessidades especiais. A *acessibilidade* para usuários (e engenheiros de software) que podem vir a ser desafiados fisicamente é um imperativo por razões éticas, legais e comerciais. Uma grande variedade de diretrizes de acessibilidade (por exemplo, [W3C03]) — muitas projetadas para aplicações Web, mas em geral aplicáveis a todos os tipos de software — dão sugestões detalhadas para projeto de interfaces que atingem níveis de acessibilidade variados. Outros (por exemplo, [App08], [Mic08]) apresentam orientações específicas para “tecnologia assistente” que lida com as necessidades daqueles com problemas visuais, auditivos, motores, de fala e de aprendizado.

Internacionalização. Os engenheiros de software e seus gerentes invariavelmente subestimam o esforço e as capacidades necessárias para criar interfaces do usuário que levem em conta as necessidades de diferentes localidades e idiomas. Muitas vezes, as interfaces são desenhadas para um país e idioma e então improvisadas para funcionar em outros. O desafio para os projetistas de interfaces é criar um software “globalizado”. Isto é, as interfaces com o usuário devem ser projetadas para atender um núcleo genérico de funcionalidade que possa ser entregue a todos que usam o software. Recursos de *localização* permitem que a interface seja personalizada para um mercado específico.

Uma grande variedade de diretrizes de internacionalização (por exemplo, [IBM03]) se encontra disponível para os engenheiros de software. Tais diretrizes tratam de ampla gama de questões de projeto (por exemplo, os layouts de tela talvez difiram em vários mercados) e problemas de implementação diferenciados (por exemplo, diferentes alfabetos talvez criem exigências especiais de atribuição de nomes e espaçamento). O padrão *Unicode* [Uni03] foi desenvolvido para tratar do intimidante desafio de gerenciar dezenas de linguagens naturais com centenas de caracteres e símbolos.

FERRAMENTAS DO SOFTWARE



Desenvolvimento de interfaces do usuário

Objetivo: Essas ferramentas permitem a um engenheiro de software criar uma sofisticada interface gráfica do usuário com relativamente pouco desenvolvimento de software personalizado. As ferramentas dão acesso a componentes reutilizáveis e tornam a criação da interface uma simples questão de selecionar capacidades predefinidas que são agrupadas usando-se a ferramenta.

Mecânica: As interfaces do usuário modernas são construídas usando-se um conjunto de componentes reutilizáveis associados a alguns componentes personalizados desenvolvidos para oferecer recursos especializados. A maioria das ferramentas para desenvolvimento de interfaces com o usuário permite que um engenheiro de software crie uma interface usando recursos de “arrastar e soltar”. Isto é, o desenvolvedor escolhe alguns de vários recursos predefinidos (por exemplo, recursos de construção de formulários, mecanismos de interação, processamento

de comandos) e insere tais recursos no conteúdo da interface a ser criada.

Ferramentas representativas:⁷

LegaSuite GUI, desenvolvida pela Seagull Software (www.seagullsoftware.com), permite a criação de GUIs baseadas em navegadores e oferece recursos para a reengenharia de interfaces antiquadas.

Motif Common Desktop Environment, desenvolvida pelo The Open Group (www.osf.org/tech/desktop/cde/), é uma interface gráfica do usuário integrada para sistemas desktop abertos. Ela oferece uma única interface gráfica padrão para o gerenciamento de dados e arquivos (a área de trabalho gráfica) e aplicações.

Altia Design 8.0, desenvolvida pela Altia (www.altia.com), é uma ferramenta para criação de GUIs em uma variedade de plataformas diferentes (por exemplo, automotiva, dispositivos portáteis, industrial).

⁷ As ferramentas aqui apresentadas não significam um aval, mas sim uma amostra dessa categoria.

11.5 PROJETO DE INTERFACES PARA WEBAPP

Toda interface do usuário — seja ela desenhada para uma WebApp, uma aplicação de software tradicional, um produto de consumo ou para um dispositivo industrial — deve apresentar as características de usabilidade discutidas neste capítulo. Dix [Dix99] argumenta que se deve projetar uma interface para WebApp de modo que responda a três perguntas primárias para o usuário final:



Se for provável que os usuários entrarão em sua WebApp em vários pontos e níveis dentro da hierarquia de conteúdo, certifique-se de desenhar cada página com recursos de navegação que levem o usuário a outros pontos de interesse.

Onde me encontro? A interface deve: (1) dar uma indicação da WebApp que foi acessada⁸ e (2) informar o usuário a sua localização na hierarquia de conteúdos.

O que posso fazer agora? A interface sempre deve auxiliar o usuário a entender suas opções atuais — que funções estão disponíveis, quais links ainda existem e qual conteúdo é relevante?

Onde estive, para onde estou indo? A interface deve facilitar a navegação. Portanto, tem de fornecer um “mapa” (implementado de maneira fácil de ser compreendido) de onde o usuário esteve e quais caminhos devem ser tomados para ir a algum outro ponto dentro da WebApp.

Uma interface WebApp eficaz deve dar respostas a cada uma dessas questões enquanto o usuário final navega pelo conteúdo e funcionalidade.

11.5.1 Princípios e diretrizes para projeto de interfaces

A interface do usuário de uma WebApp é sua “primeira impressão”. Independentemente do valor de seu conteúdo, da sofisticação de seus recursos e serviços de processamento, bem como do benefício geral da própria WebApp, uma interface malfeita desapontará o usuário potencial e talvez faça com que, de fato, procure outra opção. Devido ao grande número de WebApps concorrentes em praticamente qualquer área de aplicação, a interface tem de “atrair” imediatamente um possível usuário.

Bruce Tognozzi [Tog01] define um conjunto de características fundamentais que todas as interfaces deveriam apresentar e, ao fazer isso, estabelece uma filosofia que deveria ser seguida por todo projetista de interfaces para WebApps:

Interfaces eficazes são visualmente aparentes e “generosas”, instilando em seus usuários uma sensação de controle. Os usuários veem rapidamente o leque de opções, captam como atingir seus objetivos e realizar seus trabalhos.

Interfaces eficazes não fazem com que o usuário se preocupe com os detalhes de funcionamento interno do sistema. O trabalho é cuidadosa e continuamente salvo, com total possibilidade de o usuário desfazer qualquer atividade a qualquer momento.

Aplicações e serviços eficazes realizam o máximo de trabalho, exigindo o mínimo de informações dos usuários.

Para poder projetar interfaces para WebApp que apresentem essas características, Tognozzi [Tog01] identifica um conjunto de princípios de projeto⁹ primordiais:

Antecipação. *Uma WebApp deve ser desenhada para prever o próximo passo do usuário.* Consideremos, por exemplo, uma WebApp de apoio ao cliente desenvolvida por um fabricante de impressoras. Um usuário solicitou um objeto de conteúdo que apresente informações sobre um driver de impressora para um sistema operacional recém-lançado. O projetista da WebApp deve prever que o usuário possa vir a solicitar um download do driver e deve oferecer recursos de navegação que permitam que isso aconteça sem exigir do usuário a busca desse recurso.

“Se um site for perfeitamente utilizável, mas lhe faltar um estilo de projeto elegante e apropriado, ele estará fadado ao fracasso.”

Curt Cloninger

PONTO-CHAVE

Uma interface para WebApp de qualidade deve ser compreensível e flexível, dando ao usuário a sensação de controle.



Existe um conjunto de princípios básicos que possam ser aplicados à medida que desenhemos uma GUI?

⁸ Cada um de nós já passou pela experiência de criar um bookmark de uma página Web e ao revisita-la não ter indicação alguma do site ou do contexto para tal página Web (bem como nenhuma forma de se deslocar para outro local no site).

⁹ Os princípios de Tognozzi originais foram adaptados e estendidos para uso neste livro. Veja [Tog01] para uma discussão mais ampla sobre esses princípios.

Comunicação. *A interface deve comunicar o estado de qualquer atividade iniciada pelo usuário. A comunicação deve ser óbvia (por exemplo, uma mensagem de texto) ou sutil (por exemplo, a imagem de uma folha de papel deslocando-se pela impressora para indicar que a impressão está em andamento). A interface também deve comunicar o estado do usuário (por exemplo, a identificação do usuário) e sua localização na hierarquia de conteúdos da WebApp.*

Consistência. *O uso de controles de navegação, menus, ícones e estética (por exemplo, cor, forma, layout) devem ser consistentes em toda a WebApp. Por exemplo, se texto sublinhado na cor azul sugerir um link de navegação, o conteúdo jamais deveria incorporar texto sublinhado azul que não indique um link. Além disso, um objeto, digamos, um triângulo amarelo, usado para indicar uma mensagem de alerta antes de o usuário chamar determinada função ou ação, não deve ser usado para outras finalidades em nenhum outro ponto da WebApp. Por fim, todo recurso da interface deve responder de maneira consistente com as expectativas¹⁰ do usuário.*

Autonomia controlada. *A interface deve facilitar a movimentação do usuário pela WebApp, mas deve fazê-lo de forma que faça valer convenções de navegação estabelecidas para a aplicação. Por exemplo, a navegação em trechos de segurança da WebApp deve ser controlada por meio de identificação e senhas de usuário e não deve existir nenhum mecanismo de navegação que possibilite a um usuário evitar tais controles.*

Eficiência. *O projeto de uma WebApp e sua interface devem otimizar a eficiência de trabalho do usuário, e não a eficiência do desenvolvedor que a projeta e constrói ou o ambiente cliente/servidor que a executa. Tognozzi [Tog01] discute isso ao escrever: “Esta simples verdade é a razão de ser tão importante para todos os envolvidos em um projeto de software perceber a importância de fazer com que a produtividade do usuário seja a primeira meta e compreender a diferença vital entre construir um sistema eficiente e dar poderes a um usuário eficiente”.*

Flexibilidade. *A interface deve ser flexível o bastante para permitir que alguns usuários cumpram tarefas diretamente ao passo que outros devam explorar a WebApp de maneira um tanto aleatória. Em todos os casos, ela deve permitir ao usuário compreender onde ele se encontra e também lhe dar um recurso para desfazer erros e refazer caminhos de navegação mal escolhidos.*

Foco. *A interface para WebApp (e o conteúdo por ela apresentado) devem permanecer focados na(s) tarefa(s) do usuário em questão. Em toda hipermídia há uma tendência de direcionar o usuário a conteúdo de pouca relação. Por quê? Porque é muito fácil fazê-lo! O problema é que o usuário pode rapidamente se perder em muitas camadas de informações de apoio e perder de vista o conteúdo original que desejava de início.*

Lei de Fitt. *“O tempo para atingir um alvo é função da distância e do tamanho do alvo” [Tog01]. Baseado em um estudo realizado nos anos 1950 [Fit54], a lei de Fitt “é um método eficaz de modelar movimentos rápidos e dirigidos, em que um apêndice (como uma mão), inicia em repouso em uma posição inicial específica e se desloca para o repouso em uma área-alvo” [Zha02]. Se a sequência de seleções ou entradas padronizadas (com muitas opções diferentes na sequência) for definida por uma tarefa e usuário, a primeira seleção (por exemplo, uma seleção com o mouse) deve ser fisicamente próxima da seleção seguinte. Consideremos, por exemplo, uma interface de homepage de uma WebApp em um site de comércio eletrônico que vende produtos eletrônicos de consumo.*

Cada opção de usuário implica um conjunto de escolhas ou ações de usuário que vêm em seguida. Por exemplo, a opção “comprar um produto” requer que o usuário introduza uma categoria de produto seguida pelo nome do produto. A categoria do produto (por exemplo, equipamento de áudio, televisores, aparelhos de DVD) aparece como um menu pull-down assim que a opção “comprar um produto” for selecionada. Consequentemente, a escolha seguinte é óbvia à primeira vista (ela se encontra nas proximidades), e o tempo para reconhecê-la será desprezível.

“O melhor caminho é aquele com o menor número de passos. Diminua a distância entre o usuário e sua meta.”

Autor desconhecido

WebRef

Uma busca na Web irá revelar diversas bibliotecas disponíveis como, por exemplo, pacotes de API Java, interfaces e classes, em java.sun.com ou COM, DCOM e Type Libraries em msdn.microsoft.com.

¹⁰ Tognozzi [Tog01] observa que a única maneira de garantir que as expectativas de um usuário sejam adequadamente compreendidas é por meio de exaustivos testes com o usuário (Capítulo 20).

Se, por outro lado, a escolha aparecesse em um menu que estivesse localizado do outro lado da tela, o tempo para o usuário reconhecê-lo (e depois fazer sua escolha) seria bem mais longo.

Objeto de interface humana. *Desenvolveu-se uma vasta biblioteca de objetos de interface humana reutilizáveis para WebApps. Use-a.* Qualquer objeto de interface que possa ser “visto, ouvido, tocado ou de alguma outra forma percebido” [Tog01] por um usuário final pode ser obtido de qualquer uma das várias bibliotecas de objetos.

Redução da latência. *Em vez de fazer com que o usuário espere por alguma operação interna completar (por exemplo, baixar uma imagem complexa), a WebApp deve usar multitarefas de uma forma que deixe o usuário prosseguir com seu trabalho como se a operação tivesse sido completada.* Além de reduzir a latência, devem ser reconhecidos atrasos, de modo que o usuário entenda o que está ocorrendo. Isso inclui: (1) fornecer feedback de áudio quando uma seleção não resulte em uma ação imediata pela WebApp, (2) exibir uma animação de relógio ou barra de progresso para indicar que o processamento está em andamento e (3) fornecer algum entretenimento (por exemplo, uma apresentação de texto ou animação) enquanto ocorre processamento moroso.

Facilidade de aprendizagem. *Uma interface para WebApp deve ser projetada para minimizar o tempo de aprendizagem e, uma vez aprendida, minimizar a reaprendizagem necessária quando a WebApp for reutilizada.* Em geral, a interface deve enfatizar um projeto simples e intuitivo que organiza conteúdo e funcionalidade em categorias que são óbvias para o usuário.



Metáforas são uma excelente ideia, pois espelham a experiência do mundo real. Apenas certifique-se de que a metáfora escolhida seja bem conhecida dos usuários finais.

Metáforas. *Uma interface que usa uma metáfora de interação é mais fácil de aprender e usar, desde que a metáfora seja apropriada para a aplicação e o usuário.* A metáfora deve invocar imagens e conceitos da experiência do usuário, mas não precisa ser uma reprodução exata de uma experiência do mundo real. Por exemplo, um site de comércio eletrônico que implemente pagamento automatizado de contas para uma instituição financeira, usa uma metáfora talonário de cheques (não é de surpreender) para auxiliar o usuário a especificar e programar pagamentos de contas. Entretanto, quando um usuário “preenche” um cheque, ele não precisa introduzir o nome completo do beneficiário, mas poderá sim escolher de uma lista de beneficiários ou fazer com que o sistema o selecione baseado logo nas primeiras letras digitadas. A metáfora permanece intacta, porém o usuário obtém uma ajuda da WebApp.

Manter a integridade do artefato. *Um artefato (por exemplo, um formulário preenchido pelo usuário, uma lista especificada pelo usuário) deve ser salvo automaticamente para não ser perdido caso ocorra algum erro.* Todos já experimentamos a frustração associada a completar um enorme formulário de WebApp apenas para ter o conteúdo perdido devido a um erro (por nós cometido, cometido pela WebApp ou durante a transmissão do cliente para o servidor). Para evitar isso, uma WebApp deve ser projetada para salvar automaticamente todos os dados especificados pelo usuário. A interface deve oferecer suporte a essa função e oferecer ao usuário um mecanismo fácil para recuperar informações “perdidas”.

Legibilidade. *Todas as informações apresentadas em uma interface devem ser legíveis por jovens e idosos.* O projetista de interfaces deve dar prioridade a estilos de tipos e tamanhos de fontes legíveis e fundos coloridos que aumentem o contraste.

Acompanhar o estado de interação. *Quando apropriado, o estado da interação de usuário deve ser acompanhado e armazenado de modo que um usuário possa sair do sistema e retornar mais tarde, prosseguindo do ponto onde parou.* Em geral, podem ser projetados cookies para armazenar informações de estado. Entretanto, os cookies são uma tecnologia controversa e outras soluções de projeto talvez sejam mais aceitáveis para alguns usuários.

Navegação visível. *Uma interface para WebApp bem projetada fornece “a ilusão de que os usuários se encontram no mesmo lugar, com o trabalho sendo levado a eles” [Tog01].* Quando é usada esta abordagem, a navegação não é uma preocupação para o usuário. Ao contrário, o usuário recupera objetos de conteúdo e seleciona funções que são exibidas e executadas através da interface.

CASASEGURA

**Revisão de projeto de interfaces**

Cena: Sala de Doug Miller.

Atores: Doug Miller (gerente do grupo de engenharia de software do CasaSegura) e Vinod Raman, membro da equipe de engenharia de software do produto CasaSegura.

Conversa:

Doug: Vinod, você e a equipe tiveram a chance de revisar o protótipo de interface para comércio eletrônico **CasaSegura-Garantida.com**?

Vinod: Sim... Todos nós navegamos nele de um ponto de vista técnico e tenho um bocado de comentários. Ontem, os enviei por e-mail a Sharon [gerente da equipe de WebApps do fornecedor terceirizado para o site de comércio eletrônico CasaSegura].

Doug: Você e a Sharon podiam se reunir e discutir os pequenos detalhes... Entregue-me um resumo das questões importantes.

Vinod: Em termos gerais, eles fizeram um bom trabalho, nada de revolucionário, porém é uma interface típica para comércio eletrônico, estética aceitável, layout razoável, cobriram todas as funções importantes...

Doug (sorrindo tristemente): Mas?

Vinod: Bem, há algumas coisinhas...

Doug: Como... Vinod (**mostrando a Doug a sequência de storyboards para o protótipo de interface**): Aqui está o menu das funções principais que é apresentado na homepage:

Conheça o CasaSegura.

Descreva sua casa.

Obtenha recomendações de componentes para o CasaSegura.

Adquira um sistema CasaSegura.

Solicite suporte técnico.

O problema não é com essas funções. Todas estão corretas, mas o nível de abstração não.

Doug: Todas são funções importantes, não é mesmo?

Vinod: Sim, são, mas aí é que está o problema... É possível comprar um sistema apenas entrando com uma lista de componentes... Não há necessidade de descrever a casa se não quiser fazê-lo. Sugeriria apenas quatro opções de menu na homepage:

Conheça o CasaSegura.

Especifique o sistema CasaSegura que você precisa.

Adquira um sistema CasaSegura.

Solicite suporte técnico.

Ao selecionar **Especifique o sistema CasaSegura que você precisa**, você terá então as seguintes opções:

Selecione os componentes para o CasaSegura.

Obtenha recomendações de componentes para o CasaSegura.

Se você for um usuário experiente, escolherá componentes de um conjunto de menus pull-down classificados por sensores, câmeras, painéis de controle etc. Se precisar de ajuda, você solicitará uma recomendação, que exigirá que descreva sua casa. Imagino que seja um pouco mais lógico.

Doug: Concordo. Você conversou com a Sharon a esse respeito?

Vinod: Não, queria discutir isso primeiro com o Marketing; depois telefonaria para ela.

Nielsen e Wagner [Nie96] sugerem algumas diretrizes pragmáticas para projeto de interfaces (baseadas na experiência dos autores no reprojeito de uma importante WebApp) que são um excelente complemento aos princípios sugeridos anteriormente nesta seção:

"As pessoas têm muito pouca paciência com sites WWW mal projetados."

Jakob Nielsen e Annette Wagner

- Ler rápido em um monitor de computador é aproximadamente 25% mais lento que ler rápido em papel. Consequentemente, não force o usuário a ler toneladas de texto, particularmente quando o texto explica a operação da WebApp ou auxilia na navegação.
- Evite sinais "em construção" — um link desnecessário certamente desaparecerá.
- Os usuários preferem não "rolar" a tela. Informações importantes deveriam ser colocadas em dimensões de uma típica janela de navegação.
- Menus de navegação e barras de título devem ser desenhados de forma consistente e estar disponíveis em todas as páginas que são acessadas pelo usuário. O projeto não deve depender de funções do navegador para ajudar na navegação.
- A estética jamais deve suplantiar a funcionalidade. Por exemplo, um botão simples poderia ser uma opção de navegação melhor que um botão esteticamente charmoso, mas que é apenas uma vaga imagem ou ícone cujo objetivo não é claro.
- As opções de navegação devem ser óbvias, mesmo para o usuário casual. O usuário não deve ter de ficar procurando na tela para determinar como fazer o link com outros conteúdos ou serviços.

Uma interface bem projetada aumenta a percepção do usuário do conteúdo ou serviços fornecidos pelo site. Ela não precisa, necessariamente, ser chamativa, mas sempre deve ser bem estruturada e ergonomicamente sólida.

11.5.2 Fluxo de trabalho de projeto de interfaces para WebApps

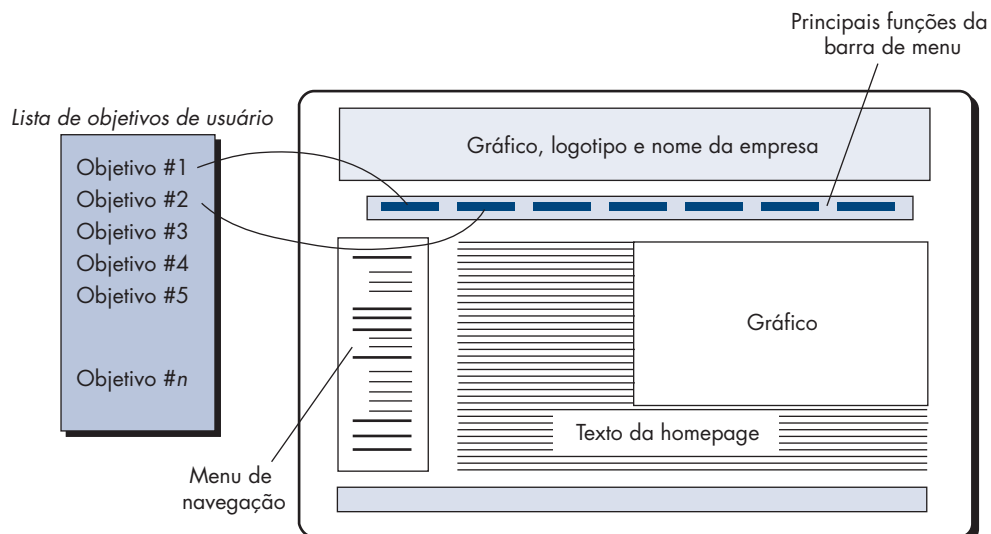
No início do capítulo citei que o projeto de interfaces do usuário começa com a identificação dos requisitos do usuário, de tarefas e dos ambientes. Uma vez que as tarefas de usuário tenham sido identificadas, cenários de usuário (casos de uso) são criados e analisados para definir um conjunto de ações e objetos de interface.

Informações contidas no modelo de requisitos formam a base para a criação de um layout de tela que represente o design gráfico e o posicionamento de ícones, a definição de texto de tela descritivo, a especificação e a colocação de títulos para as janelas, bem como a especificação de itens de menu principais e secundários. São usadas então ferramentas para prototipagem e, por fim, a implementação do modelo de projeto para interface. As tarefas a seguir representam um fluxo de trabalho rudimentar no projeto de interfaces para WebApps:

- 1. Revise as informações contidas no modelo de requisitos e refine conforme necessário.**
- 2. Desenvolva um esboço do layout para interfaces WebApp.** Um protótipo de interface (incluindo seu layout) poderia ter sido desenvolvido como parte da atividade de modelagem de requisitos. Se o layout já existir, deve ser revisto e refinado conforme necessário. Se o layout da interface não tiver sido desenvolvido, devemos trabalhar com os interessados para desenvolvê-lo nesta oportunidade. Um esboço de layout preliminar é mostrado na Figura 11.4.
- 3. Mapeie os objetivos do usuário em ações de interface específicas.** Para a grande maioria das WebApps, o usuário terá um conjunto relativamente pequeno de objetivos primários. Esses devem ser mapeados em ações de interface específicas conforme mostra a Figura 11.4. Em essência, temos de responder a seguinte pergunta: “Como a interface habilita o usuário a cumprir cada objetivo?”.
- 4. Defina um conjunto de tarefas de usuário que está associado a cada ação.** Cada ação de interface (por exemplo, “comprar um produto”) está associada a um conjunto de tarefas de usuário. Tais tarefas foram identificadas durante a modelagem de requisitos. Durante o projeto, devem ser mapeadas em interações específicas que englobam questões de navegação, objetos de conteúdo e funções WebApp.

FIGURA 11.4

Mapeamento dos objetivos de usuário em ações de interface



5. **Imagens de tela storyboard para cada ação de interface.** À medida que cada ação é considerada, uma sequência de imagens de storyboard (imagens de tela) deve ser criada para representar como a interface responde à interação com o usuário. Devem-se identificar objetos de conteúdo (mesmo que ainda não tenham sido projetados e desenvolvidos), a funcionalidade da WebApp deve ser mostrada e links de navegação devem ser indicados.
6. **Refine o layout de interface e storyboards usando entrada do projeto estético.** Na maioria dos casos, você será responsável por layout e storyboarding preliminar, mas o aspecto estético para um importante site comercial em geral é desenvolvido por profissionais artísticos e não por técnicos. O projeto estético (Capítulo 13) é integrado com o trabalho realizado pelo projetista de interface.
7. **Identifique objetos de interface com o usuário necessários para implementar a interface.** Essa tarefa poderia exigir uma busca em uma biblioteca de objetos para encontrar aqueles objetos (classes) reutilizáveis apropriados para a interface para WebApp. Além disso, quaisquer classes personalizadas são especificadas neste momento.
8. **Desenvolva uma representação procedural da interação do usuário com a interface.** Essa tarefa opcional usa diagramas de sequências UML e/ou diagramas de atividades (Apêndice 1) para representar o fluxo de atividades (e decisões) que ocorrem à medida que o usuário interage com a WebApp.
9. **Desenvolva uma representação comportamental da interface.** Essa tarefa opcional faz uso de diagramas de estados UML (Apêndice 1) para representar transições de estados e os eventos que os provocam. São definidos mecanismos de controle (isto é, os objetos e as ações disponíveis para o usuário alterar um estado da WebApp).
10. **Descreva o layout da interface para cada estado.** Usando as informações de projeto desenvolvidas nas Tarefas 2 e 5, associe a layout ou imagens de tela específicos a cada estado da WebApp descrito na Tarefa 8.
11. **Refine e revise o modelo de projeto de interface.** A revisão da interface deve se concentrar na usabilidade.

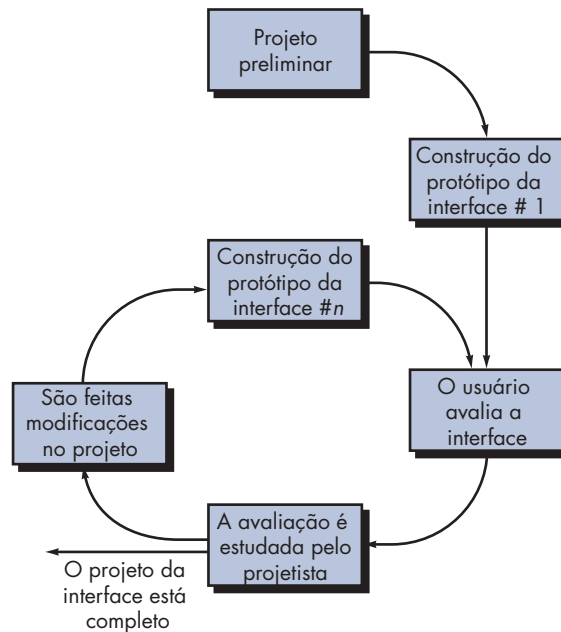
É importante notar que o conjunto final de tarefas escolhido deve ser adaptado às exigências especiais da aplicação a ser construída.

11.6 AVALIAÇÃO DE PROJETO

Assim que tiver criado um protótipo operacional de interface do usuário, ele deve ser avaliado para determinar se atende os requisitos do usuário. A avaliação pode abranger um espectro de formalidade que vai de um *test-drive* informal em que um usuário fornece feedback imediato ao estudo formalmente projetado que usa os métodos estatísticos para a avaliação de questionários preenchidos por uma população de usuários finais.

O ciclo de avaliação de interfaces do usuário assume a forma indicada na Figura 11.5. Após a finalização do modelo de projeto, cria-se um protótipo de primeiro nível. O protótipo é avaliado pelo usuário¹¹, que nos fornece comentários diretos sobre a eficácia da interface. Além disso, se forem usadas técnicas de avaliação formais (por exemplo, questionários, formulários de avaliação), podemos extrair informações desses dados (por exemplo, 80% de todos os usuários não gostam do mecanismo para salvar arquivos de dados). São feitas modificações de projeto baseadas nas informações fornecidas pelos usuários e é criado o protótipo do nível seguinte. O ciclo de avaliação continua até que nenhuma modificação adicional seja necessária para o projeto de interfaces.

11 É importante notar que os especialistas em ergonomia e projeto de interfaces também podem realizar revisões da interface. Essas revisões são denominadas *avaliações heurísticas* ou *revisões cognitivas*.

FIGURA 11.5**O ciclo de avaliação de projeto de interfaces**

A abordagem de prototipagem é efetiva, mas é possível avaliar a qualidade de uma interface do usuário antes de um protótipo ser construído? Se forem identificados e corrigidos possíveis problemas precocemente, o número de laços realizados no ciclo de avaliação será reduzido, e o tempo de desenvolvimento será abreviado. Se um modelo de projeto da interface tiver sido criado, uma série de critérios de avaliação [Mor81] poderá ser aplicada durante revisões de projeto precoces:

1. A duração e a complexidade do modelo de requisitos ou a especificação por escrito do sistema e sua interface dão uma indicação do volume de aprendizado exigido dos usuários do sistema.
2. O número de tarefas de usuário especificado e o número médio de ações por tarefa indicam o tempo de interação e a eficiência geral do sistema.
3. O número de ações, tarefas e estados do sistema indicados pelo modelo de projeto sugerem a carga de memória necessária por parte dos usuários do sistema.
4. O estilo da interface, recursos de ajuda e o protocolo de tratamento de erros dão uma indicação geral da complexidade da interface e do grau de aceitação por parte dos usuários.

Assim que o primeiro protótipo tiver sido construído, podemos coletar uma variedade de dados qualitativos e quantitativos que irão auxiliar na avaliação da interface. Para a coleta de dados qualitativos, podemos distribuir questionários aos usuários do protótipo. Entre as perguntas a ser feitas, temos: (1) respostas simples SIM/NÃO, (2) respostas numéricas, (3) resposta em padrão de escala (subjetiva), (4) escalas de Likert (como, por exemplo, concordo plenamente, concordo parcialmente), (5) resposta porcentual (subjetiva) ou (6) aberta.

Se forem desejados dados quantitativos, pode ser realizada uma espécie de análise de estudo de tempos. Os usuários são observados durante a interação, e dados — como uma série de tarefas corretamente completadas durante um período de tempo padrão, frequência de ações, sequência de ações, tempo gasto “observando” a tela, número e tipos de erros, tempo de recuperação de erros, tempo gasto usando o sistema de ajuda e o número de referências de ajuda por período de tempo padrão — são coletados e usados como um guia para modificação da interface.

Uma discussão completa dos métodos de avaliação de interfaces do usuário está fora do escopo deste livro. Para maiores informações, veja [Hac98] e [Sto05].

11.7 RESUMO

A interface do usuário é discutivelmente o elemento mais importante de um produto ou sistema computacional. Se a interface for mal projetada, a capacidade de o usuário aproveitar todo o poder computacional e conteúdo de informações de uma aplicação pode ser seriamente afetada. Na realidade, uma interface fraca pode fazer com que uma aplicação, em outros aspectos bem projetada e solidamente implementada, falhe.

Três importantes princípios orientam o projeto de interfaces do usuário eficazes: (1) deixar o usuário no comando, (2) reduzir a carga de memória do usuário e (3) tornar a interface consistente. Para alcançar uma interface que observe esses princípios, deve ser realizado um processo de projeto organizado.

O desenvolvimento de uma interface do usuário começa com uma série de tarefas de análise. A análise dos usuários define os perfis de vários usuários finais e é reunida com base em uma série de fontes técnicas e comerciais. A análise de tarefas define as tarefas e ações de usuários usando uma abordagem de refinamento ou orientada a objetos, aplicando casos de uso, elaboração de tarefa e objetos, análise de fluxos de trabalho e representações hierárquicas de tarefas para entender completamente a interação homem-computador. A análise do ambiente identifica as estruturas físicas e sociais em que a interface deve operar.

Uma vez identificadas as tarefas, são criados e analisados cenários de usuário para definir um conjunto de ações e objetos de interface. Isso fornece uma base para a criação de um layout de tela que represente o design gráfico e o posicionamento de ícones, a definição de texto de tela descritivo, a especificação e a colocação de títulos para as janelas, bem como a especificação de itens de menu principais e secundários. Questões de projeto como tempo de resposta, estrutura de comandos e ações, tratamento de erros e recursos de ajuda são consideradas à medida que o modelo de projeto é refinado. Uma grande variedade de ferramentas de implementação é usada para construir um protótipo para avaliação por parte do usuário.

Assim como ocorre no projeto de interfaces para software convencional, o projeto de interfaces para WebApps descreve a estrutura e a organização de uma interface do usuário e abrange a representação do layout de tela, a definição dos modos de interação e a descrição de mecanismos de navegação. Um conjunto de princípios para o projeto de interfaces e um fluxo de trabalho para projeto de interfaces orientam o projetista de WebApps quando o layout e mecanismos de controle da interface são desenhados.

A interface do usuário é a janela para o software. Em muitos casos, ela molda a percepção do usuário quanto à qualidade de um sistema. Se a “janela” for embaçada, ondulada ou quebrada, o usuário poderá rejeitar um sistema computacional que de outra forma seria considerado poderoso.

PROBLEMAS E PONTOS A PONDERAR

11.1. Descreva a pior interface com a qual você já tenha trabalhado até hoje e critique-a em relação aos conceitos introduzidos neste capítulo. Descreva a melhor interface com a qual já tenha trabalhado até hoje e critique-a em relação aos conceitos introduzidos neste capítulo.

11.2. Desenvolva mais dois princípios de projeto que “deixem o usuário no comando”.

11.3. Desenvolva mais dois princípios de projeto que “reduzam a carga de memória do usuário”.

11.4. Desenvolva mais dois princípios de projeto que “tornem a interface consistente”.

11.5. Considere uma das seguintes aplicações interativas (ou uma aplicação designada pelo seu professor):

- a. Um sistema de editoração eletrônica
- b. Um sistema de projeto auxiliado por computador
- c. Um sistema de projeto de interiores (conforme descrito na Seção 11.3.2)
- d. Um sistema automatizado de matrícula de uma universidade
- e. Um sistema de gerenciamento de bibliotecas
- f. Uma urna eletrônica baseada na Internet para eleições públicas
- g. Um sistema de *home banking*
- h. Uma aplicação interativa designada pelo seu professor

Desenvolva um modelo de usuário, modelo de projeto, modelo mental e um modelo de implementação para qualquer um desses sistemas.

11.6. Realize uma análise detalhada das tarefas para qualquer um dos sistemas enumerados no Problema 11.5. Use uma abordagem de refinamento ou orientada a objetos.

11.7. Acrescente pelo menos cinco outras questões à lista desenvolvida para análise de conteúdo da Seção 11.3.3.

11.8. Continuando o Problema 11.5, defina objetos e ações de interface para a aplicação que você escolheu. Identifique cada tipo de objeto.

11.9. Desenvolva um conjunto de layouts de tela com uma definição de itens de menu primários e secundários para o sistema escolhido no Problema 11.5.

11.10. Desenvolva um conjunto de layouts de tela com uma definição de itens de menu primários e secundários para o sistema *CasaSegura*. Você pode optar por uma abordagem diferente daquela indicada para o layout de tela da Figura 11.3.

11.11. Descreva sua abordagem para recursos de ajuda ao usuário para o modelo de projeto de análise de tarefas e para a análise de tarefas realizada como parte dos Problemas 11.5 a 11.8.

11.12. Dê alguns exemplos que ilustrem por que a variabilidade no tempo de resposta pode ser um problema.

11.13. Desenvolva uma abordagem que integraria automaticamente mensagens de erro e um recurso de ajuda ao usuário. Isto é, o sistema reconheceria automaticamente o tipo de erro e forneceria uma janela de ajuda com sugestões para corrigi-lo. Realize um projeto de software razoavelmente completo que considere estruturas de dados e algoritmos apropriados.

11.14. Desenvolva um questionário para avaliação de interfaces contendo 20 perguntas genéricas que se aplicariam à maioria das interfaces. Faça com que 10 companheiros de classe completem o questionário para um sistema interativo que todos usarão. Sintetize os resultados e relate-os à sua classe.

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

Embora este livro não seja especificamente sobre interfaces homem-computador, muito daquilo que Donald Norman (*The Design of Everyday Things*, reedição, Currency/Doubleday, 1990) tem a dizer sobre a psicologia de projeto efetivo se aplica a uma interface do usuário. É uma leitura recomendada a todos que estejam empenhados em desenvolver um projeto de interface de alta qualidade.

As interfaces gráficas do usuário são onipresentes no mundo moderno da computação. Seja ela um caixa eletrônico, um telefone celular, um painel eletrônico de comandos em um automóvel, um site ou uma aplicação comercial, a interface do usuário oferece uma janela para o software. É por essa razão que há inúmeros livros que tratam de projeto de interfaces. Butow (*User Interface Design for Mere Mortals*, Addison-Wesley, 2007), Galitz (*The Essential Guide to User Interface Design*, 3. ed., Wiley, 2007), Lehtonen e seus colegas (*Personal Content Experience: Managing Digital Life in the Mobile Age*, Wiley-Interscience, 2007), Cooper e seus

colegas (*About Face 3: The Essentials of Interaction Design*, 3. ed., Wiley, 2007), Ballard (*Designing the Mobile User Experience*, Wiley, 2007), Nielsen (*Coordinating User Interfaces for Consistency*, Morgan-Kaufmann, 2006), Lauesen (*User Interface Design: A Software Engineering Perspective*, Addison-Wesley, 2005), Barfield (*The User Interface: Concepts and Design*, Bosko Books, 2004) todos discutem temas como usabilidade, conceitos, princípios e técnicas de projeto para interfaces do usuário e contêm muitos exemplos úteis.

Livros mais antigos como os de Beyer e Holtzblatt (*Contextual Design: A Customer Centered Approach to Systems Design*, Morgan-Kaufmann, 2002), Raskin (*The Humane Interface*, Addison-Wesley, 2000), Constantine e Lockwood (*Software for Use*, ACM Press, 1999) e Mayhew (*The Usability Engineering Lifecycle*, Morgan-Kaufmann, 1999) apresentam tratados que fornecem diretrizes e princípios de projeto adicionais, bem como sugestões para levantamento de necessidades, modelagem de projeto, implementação e testes de interfaces. Johnson (*GUI Bloopers: Don'ts and Do's for Software Developers and Web Designers*, Morgan-Kaufmann, 2000) dá uma útil orientação para aqueles que aprendem mais efetivamente por meio do exame de contraexemplos. Um agradável livro de Cooper (*The Inmates Are Running the Asylum*, Sams Publishing, 1999) discute por que os produtos de alta tecnologia nos deixam loucos e como projetar algum deles que não tenham esse efeito.

Uma ampla gama de fontes de informação sobre projeto de interfaces se encontra à disposição na Internet. Uma lista atualizada de referências na Web, relevante para o projeto de interfaces, pode ser encontrada no site **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.